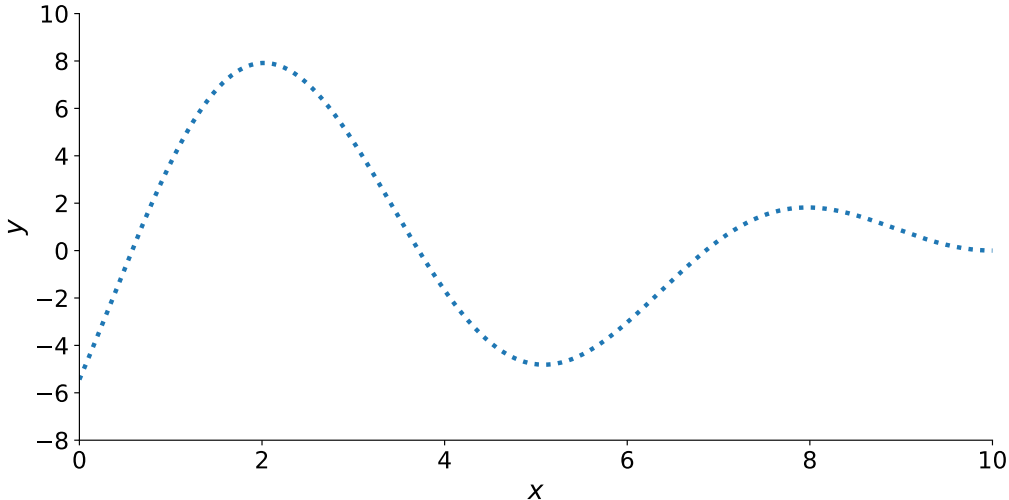
Magíster en Estadística, PUCV

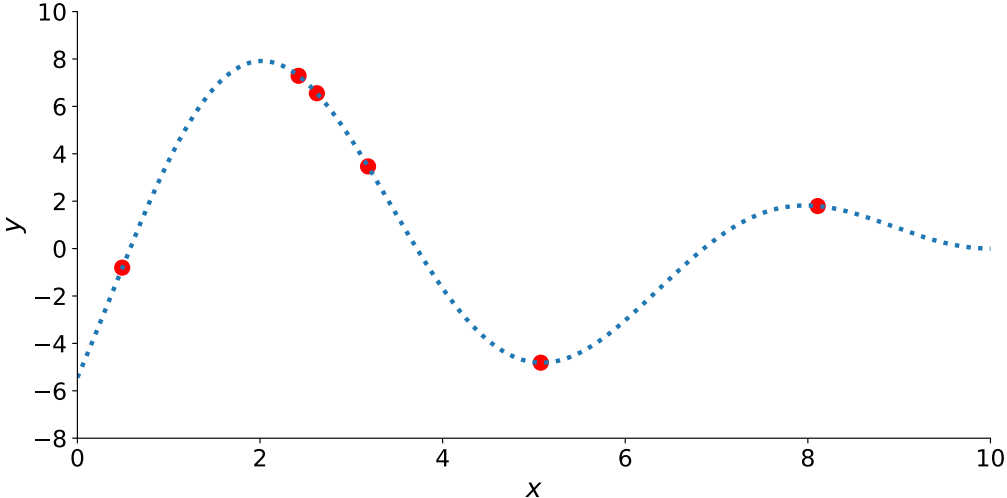
Gaussian processes

Alejandro Veloz

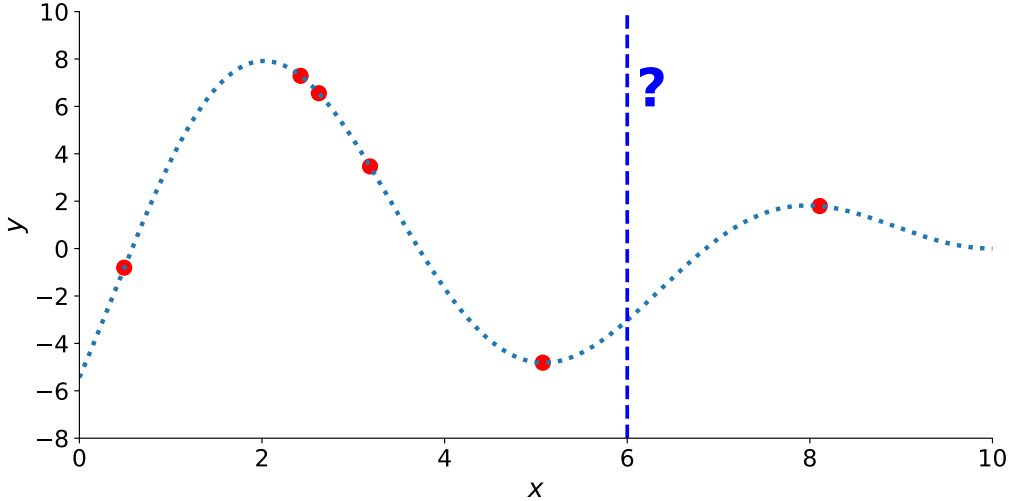
Motivation: non-linear regression



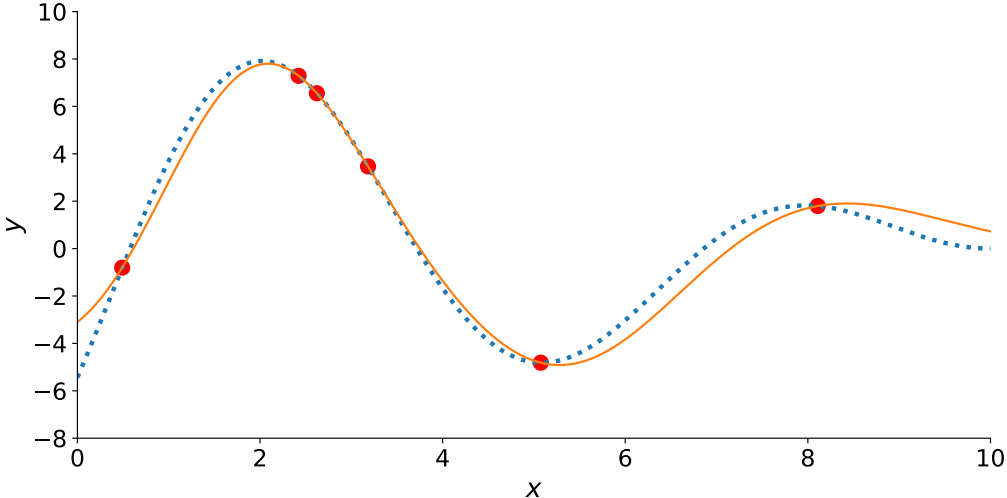
Motivation: non-linear regression



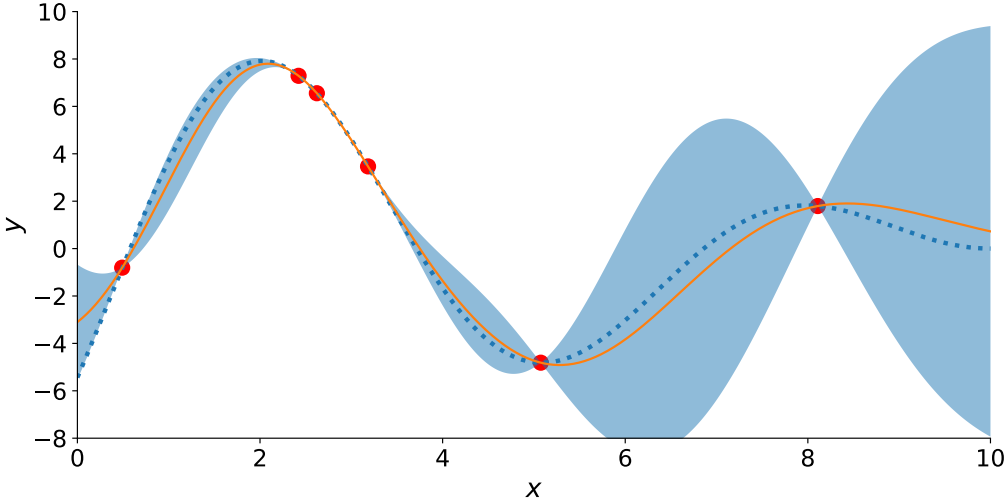
Motivation: non-linear regression



Motivation: non-linear regression



Motivation: non-linear regression



Outline

From gaussian distribution to Gaussian processes

Gaussian processes for regression

Covariance functions

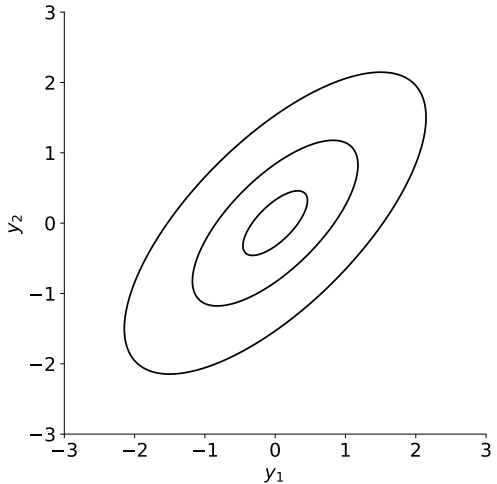
Applications

Resources

Gaussian distribution

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

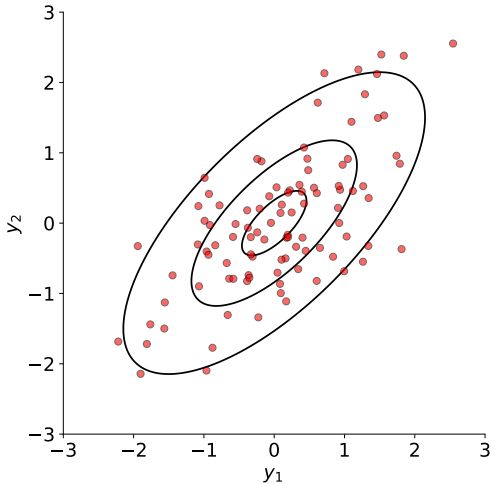
$$\Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



Gaussian distribution

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

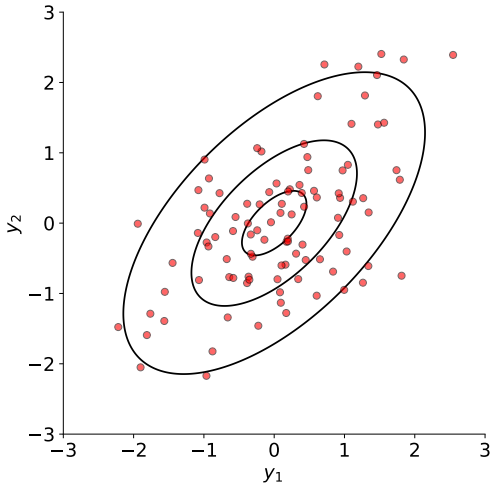
$$\Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



Gaussian distribution

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

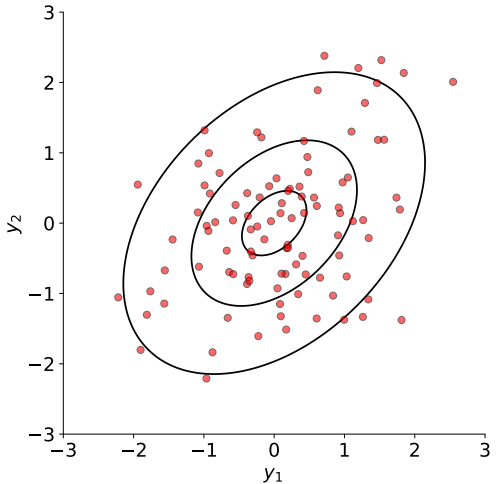
$$\Sigma = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}$$



Gaussian distribution

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

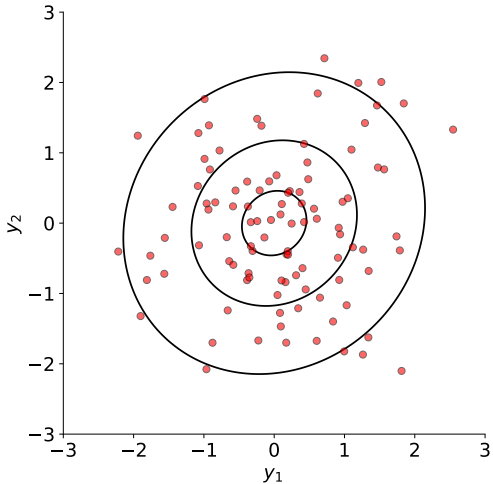
$$\Sigma = \begin{bmatrix} 1 & 0.4 \\ 0.4 & 1 \end{bmatrix}$$



Gaussian distribution

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

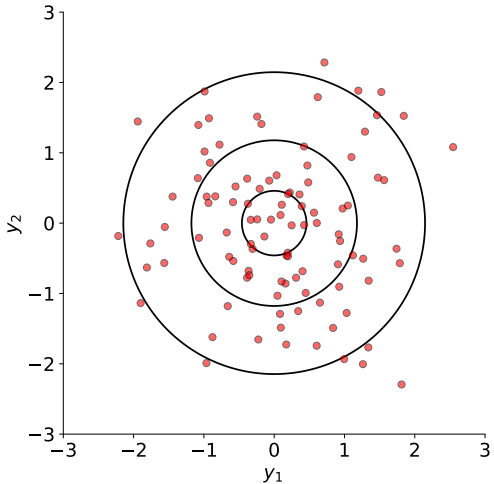
$$\Sigma = \begin{bmatrix} 1 & 0.1 \\ 0.1 & 1 \end{bmatrix}$$



Gaussian distribution

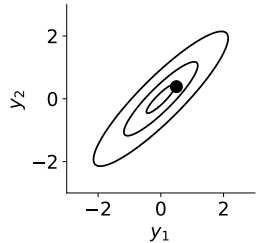
$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^{\mathsf{T}}\Sigma^{-1}\mathbf{y}\right)$$

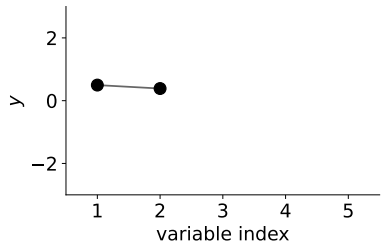
$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



New visualization

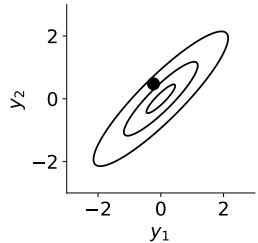
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

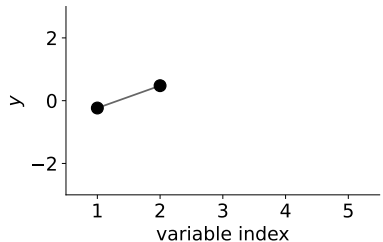




New visualization

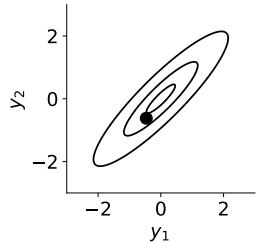
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

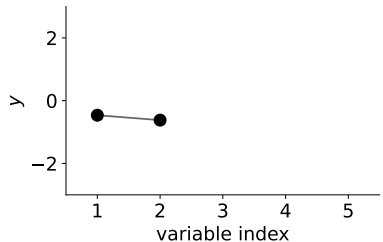




New visualization

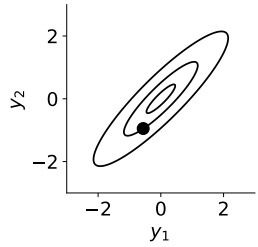
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

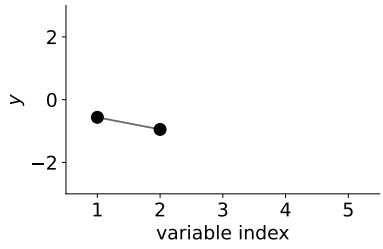




New visualization

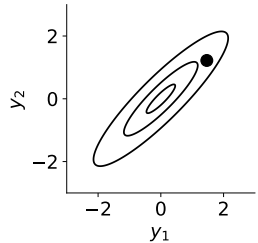
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

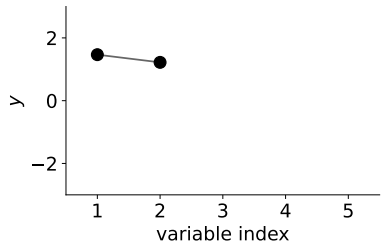




New visualization

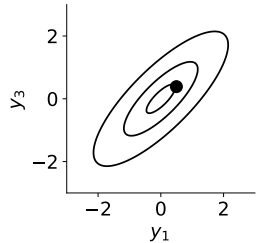
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

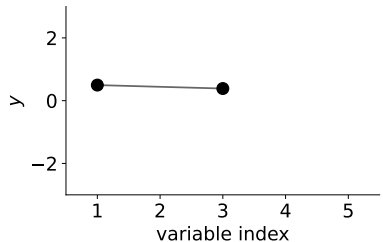




New visualization

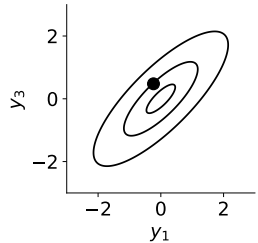
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

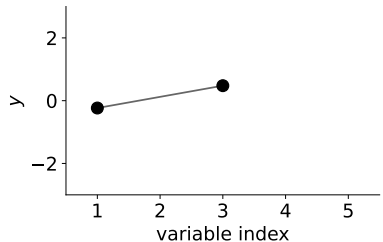




New visualization

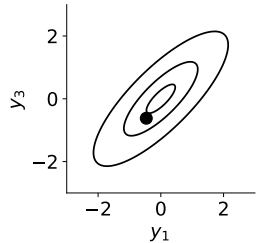
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

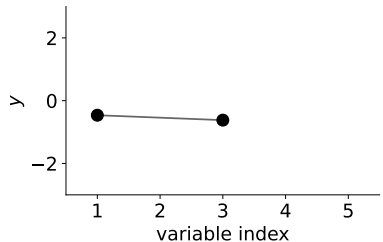




New visualization

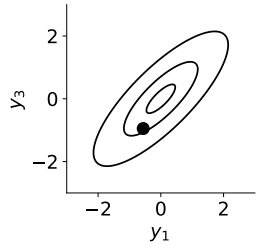
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

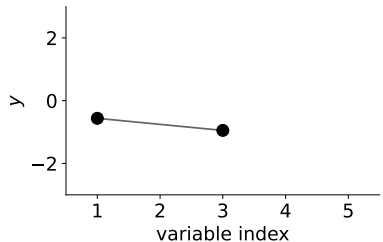




New visualization

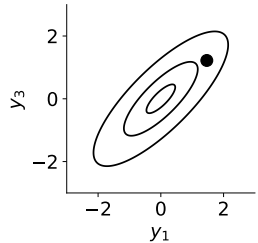
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

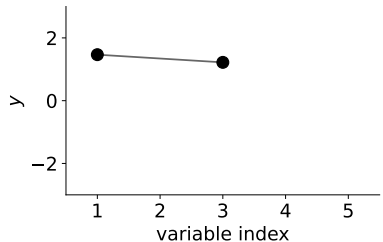




New visualization

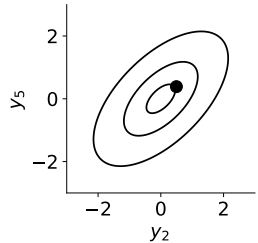
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

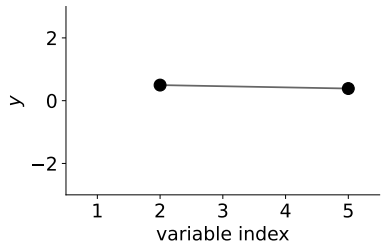




New visualization

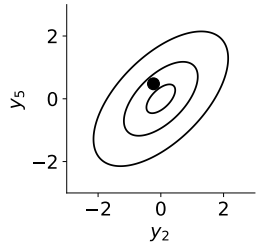
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

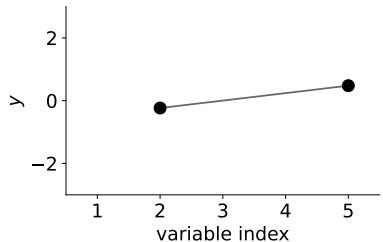




New visualization

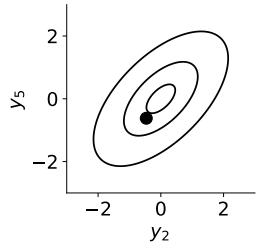
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

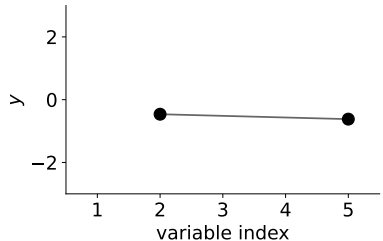




New visualization

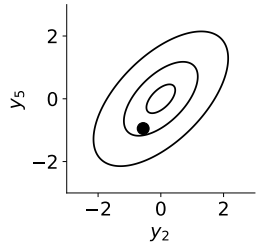
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

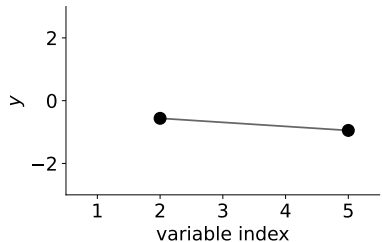




New visualization

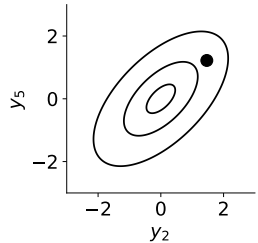
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

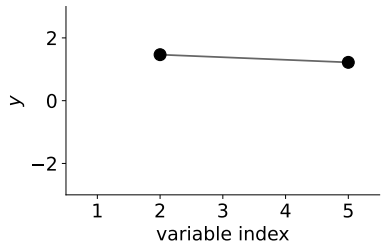




New visualization

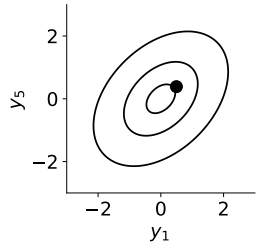
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

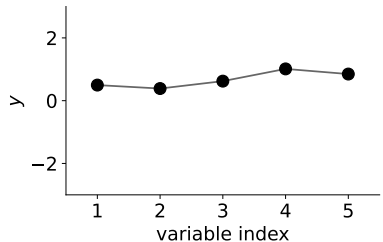




New visualization

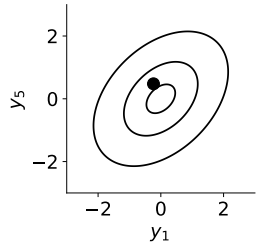
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

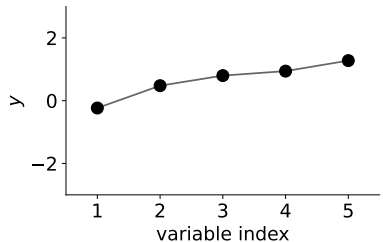




New visualization

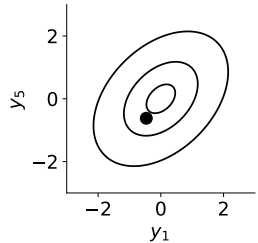
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

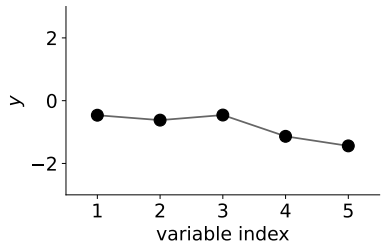




New visualization

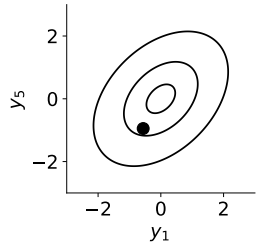
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

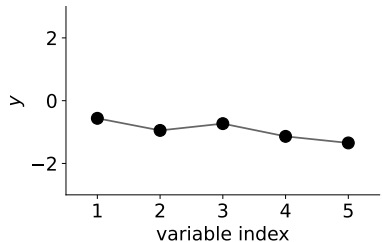




New visualization

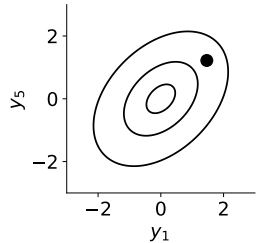
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

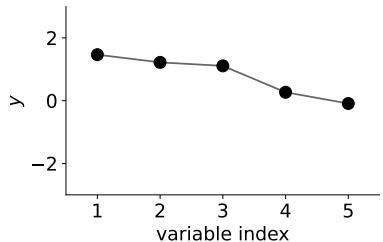




New visualization

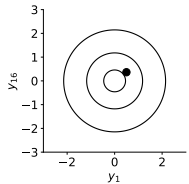
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.6 & 0.4 \\ 0.9 & 1 & 0.9 & 0.8 & 0.6 \\ 0.8 & 0.9 & 1 & 0.9 & 0.8 \\ 0.6 & 0.8 & 0.9 & 1 & 0.9 \\ 0.4 & 0.6 & 0.8 & 0.9 & 1 \end{bmatrix}$$

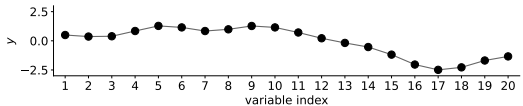




New visualization

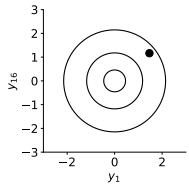
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & 0 & \dots \\ 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & \dots \\ 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & \dots \\ 0.3 & 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

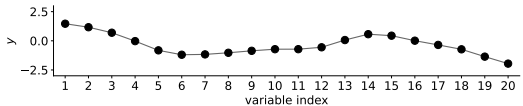




New visualization

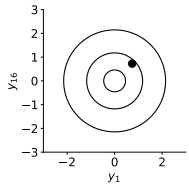
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & 0 & \dots \\ 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & \dots \\ 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & \dots \\ 0.3 & 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

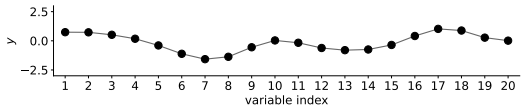




New visualization

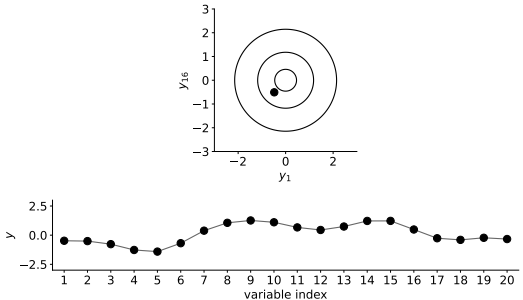
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & 0 & \dots \\ 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & \dots \\ 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & \dots \\ 0.3 & 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$





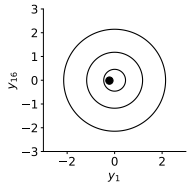
New visualization

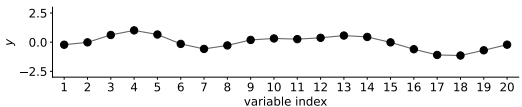
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & 0 & \dots \\ 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & \dots \\ 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & \dots \\ 0.3 & 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$



New visualization

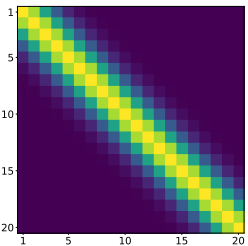
$$\Sigma = \begin{bmatrix} 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & 0 & \dots \\ 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & 0 & \dots \\ 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & 0 & \dots \\ 0.3 & 0.6 & 0.9 & 1 & 0.9 & 0.6 & 0.3 & 0.1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

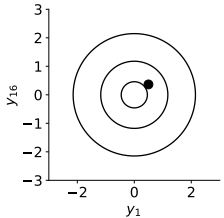


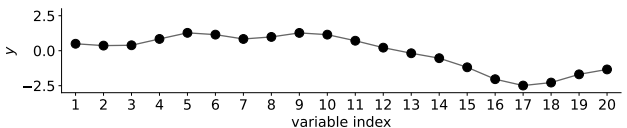


New visualization

$\Sigma =$

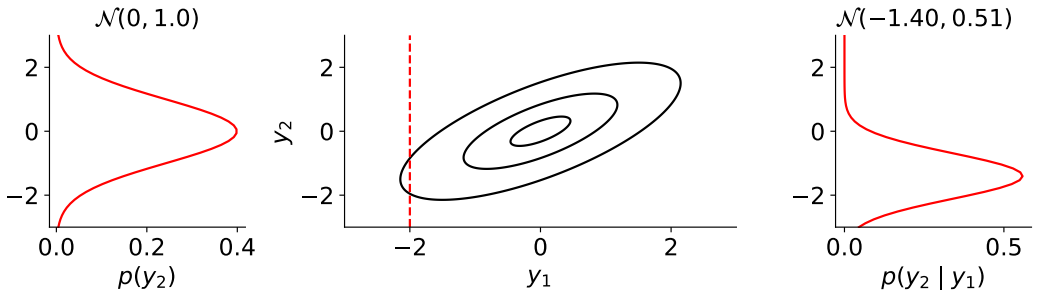






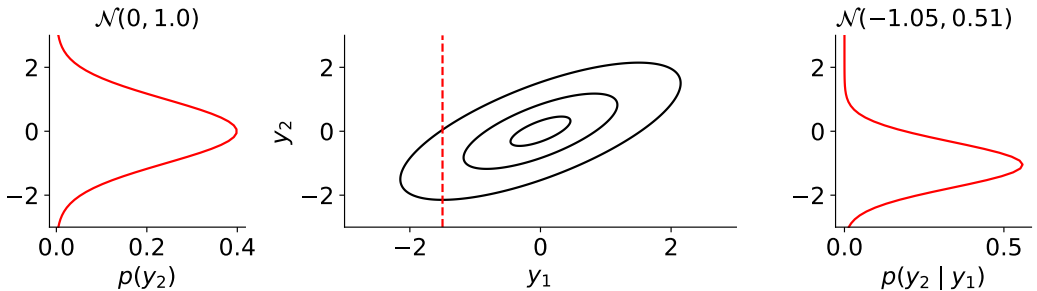
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



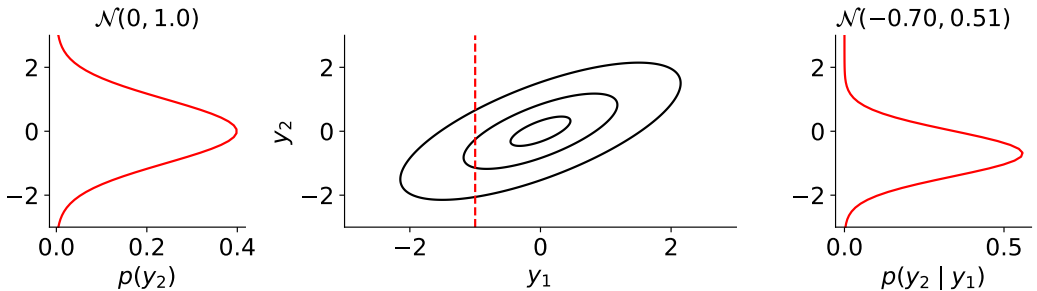
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



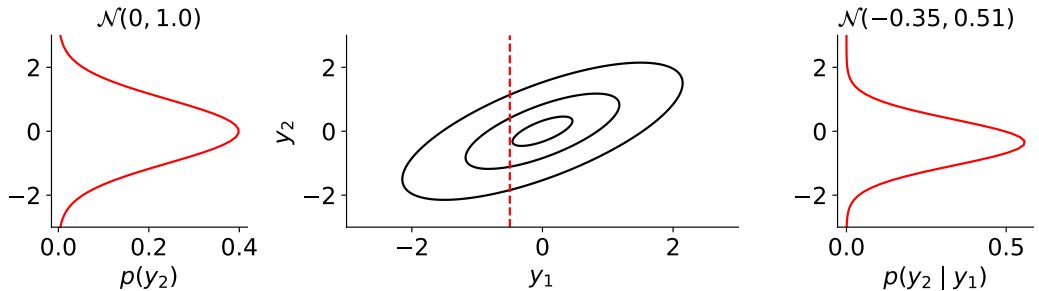
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



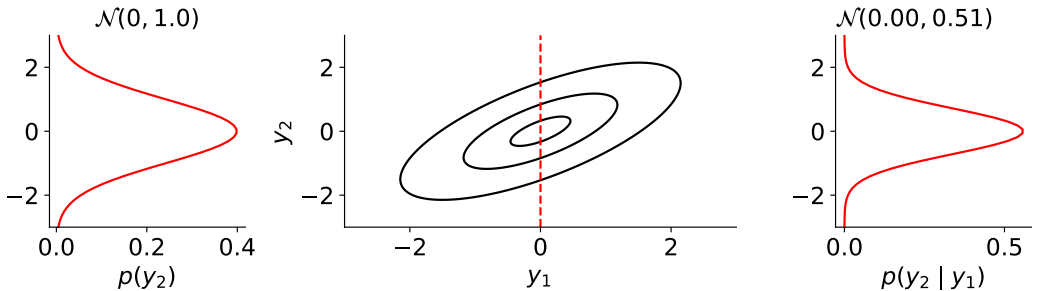
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



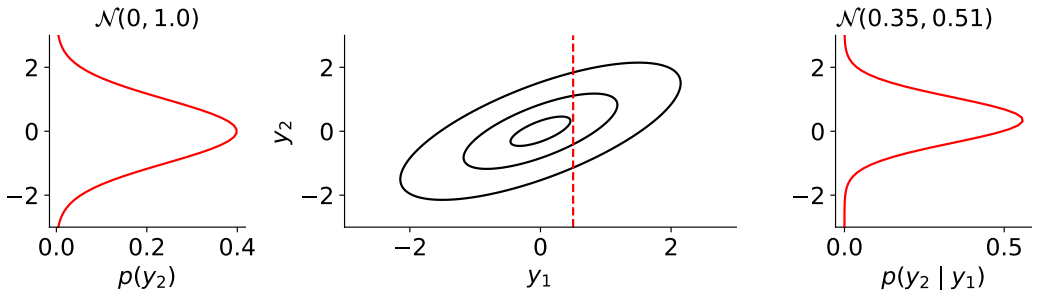
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



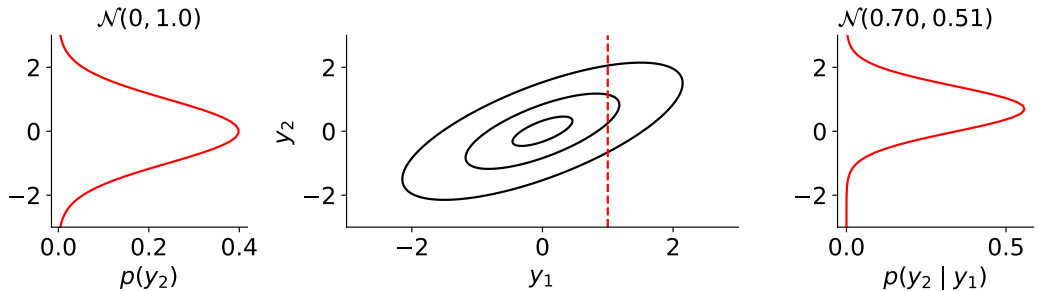
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



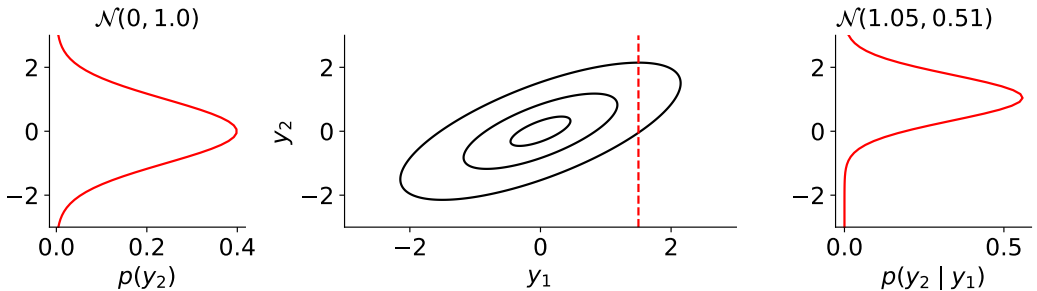
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



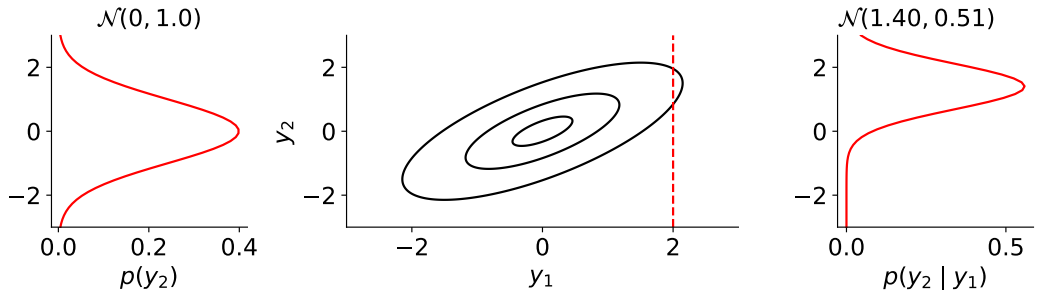
Marginalization and conditioning

$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\intercal \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



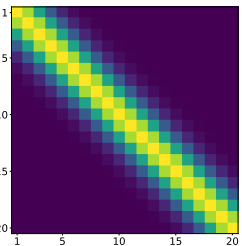
Marginalization and conditioning

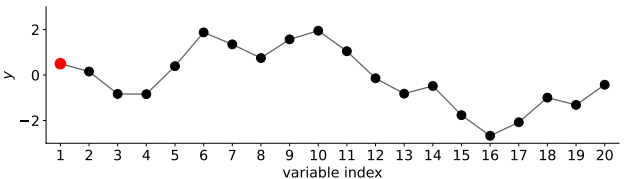
$$p(\mathbf{y} \mid \Sigma) \propto \exp\left(-\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y}\right) \qquad \Sigma = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}$$



Marginalization and conditioning

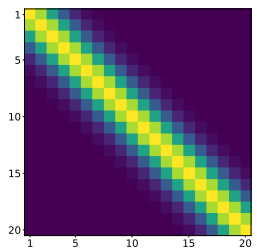
$$\Sigma =$$

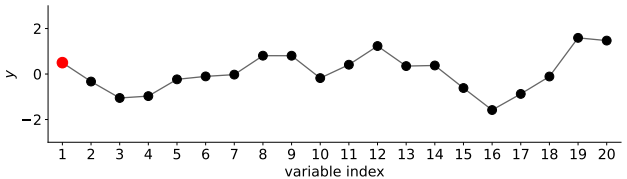




Marginalization and conditioning

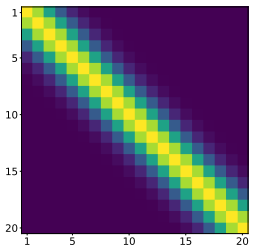
$$\Sigma =$$

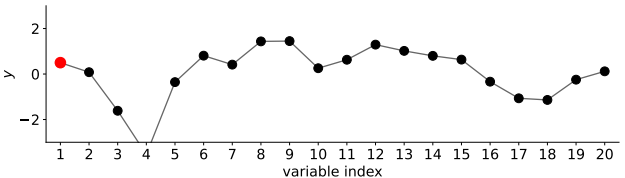




Marginalization and conditioning

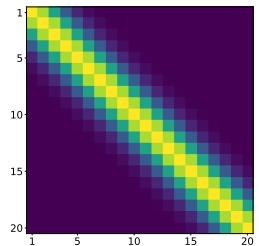
$$\Sigma =$$

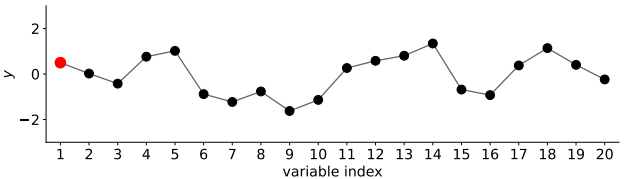




Marginalization and conditioning

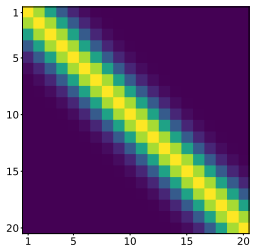
$$\Sigma =$$

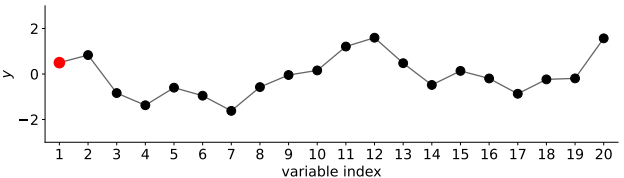




Marginalization and conditioning

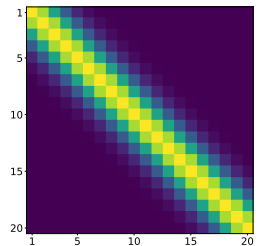
$$\Sigma =$$

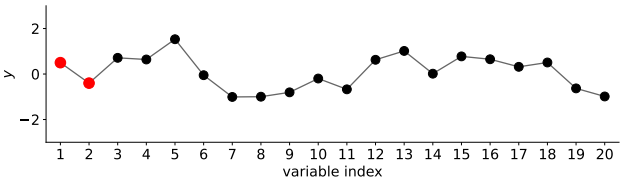




Marginalization and conditioning

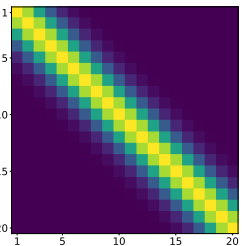
$$\Sigma =$$

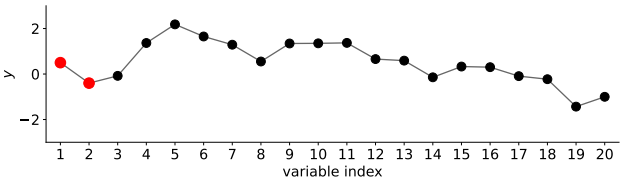




Marginalization and conditioning

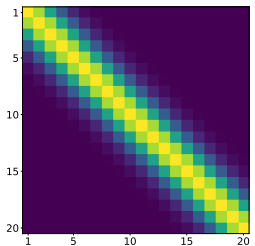
$$\Sigma =$$

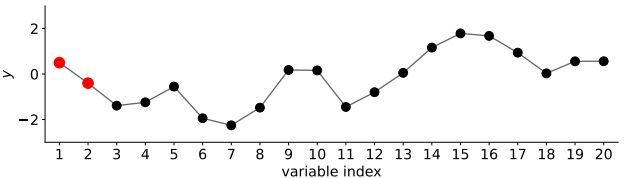




Marginalization and conditioning

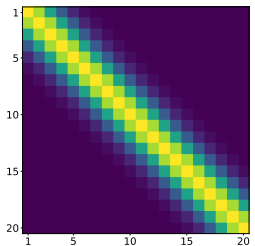
$$\Sigma =$$

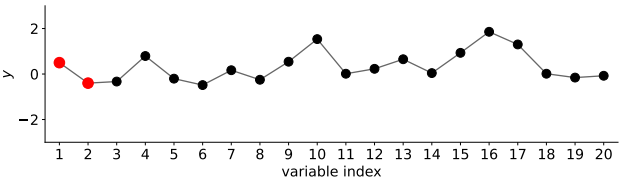




Marginalization and conditioning

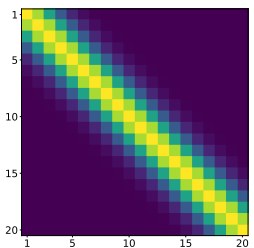
$$\Sigma =$$

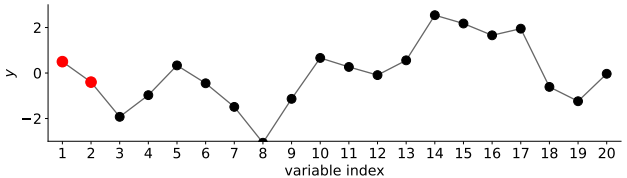




Marginalization and conditioning

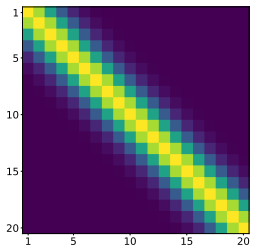
$$\Sigma =$$

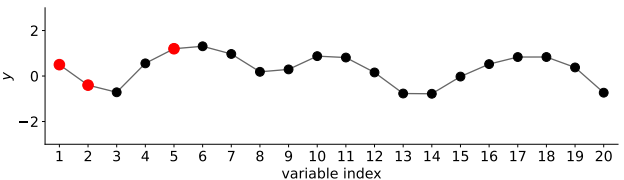




Marginalization and conditioning

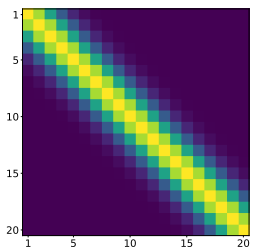
$$\Sigma =$$

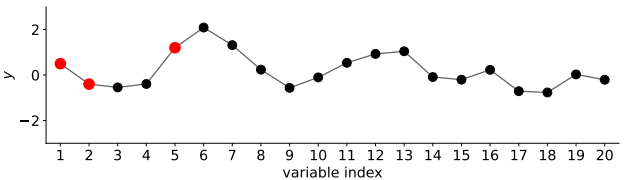




Marginalization and conditioning

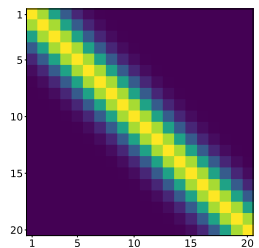
$$\Sigma =$$

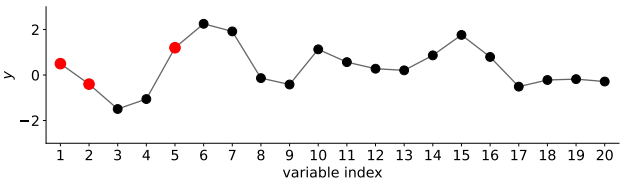




Marginalization and conditioning

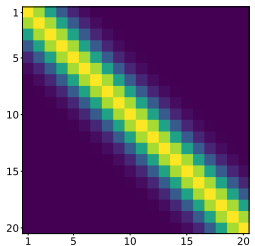
$$\Sigma =$$

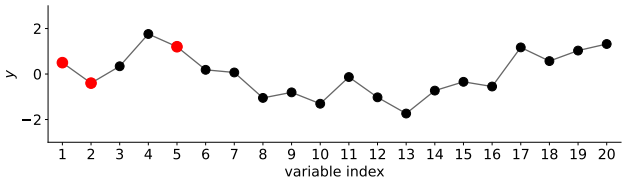




Marginalization and conditioning

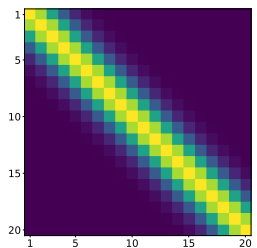
$$\Sigma =$$

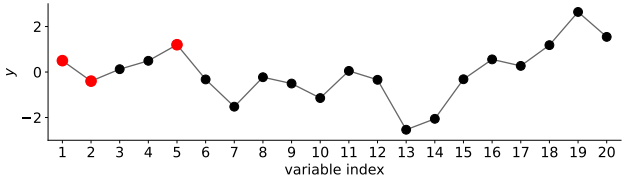




Marginalization and conditioning

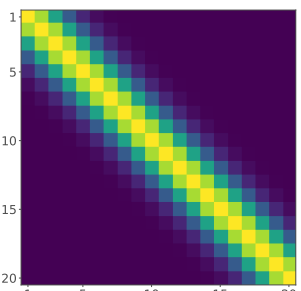
$$\Sigma =$$

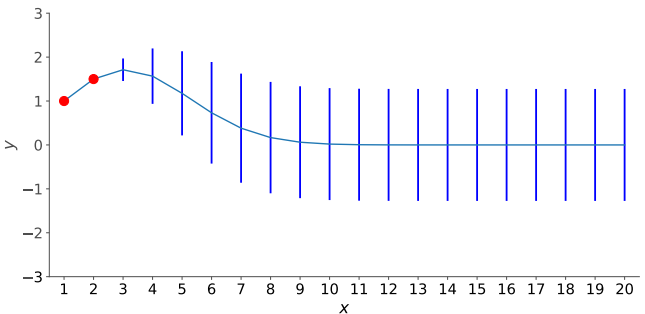




Regression using Gaussians

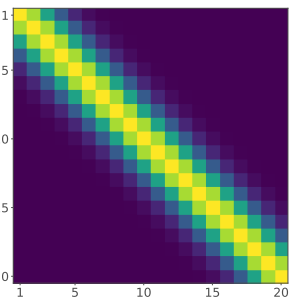
$$\Sigma =$$

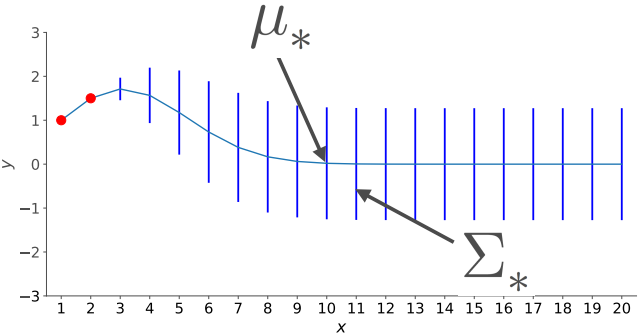




Regression using Gaussians

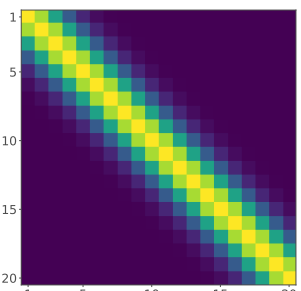
$$\Sigma =$$

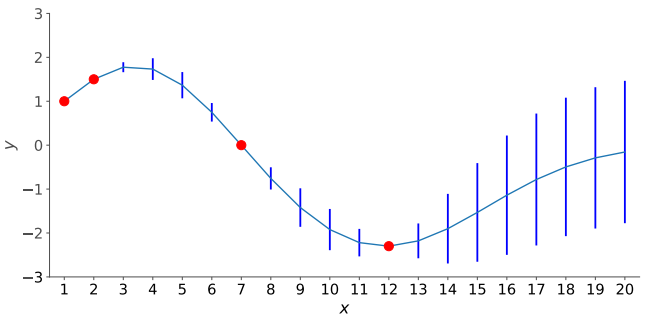




Regression using Gaussians

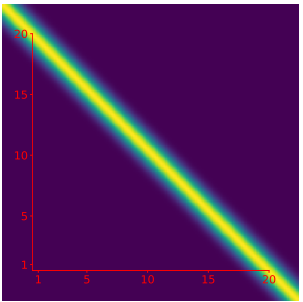
$$\Sigma =$$

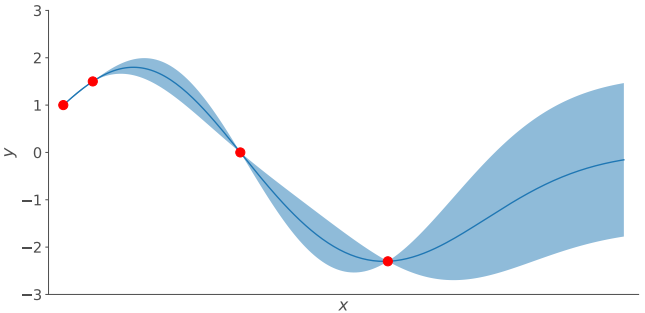




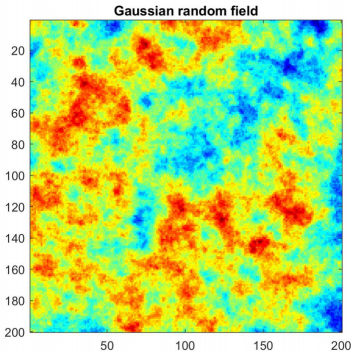
Regression using Gaussians

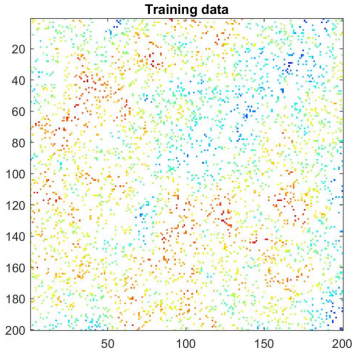
$$\Sigma =$$

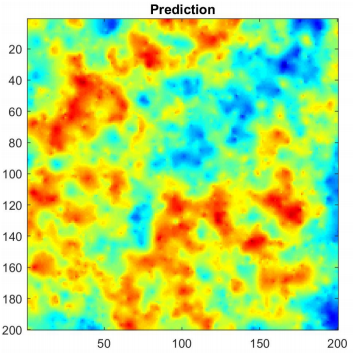




Regression using Gaussians







Gaussian processes in machine learning

- Gaussian processes (GPs), also known as Gaussian random fields.
- Originating in geostatistics, they were introduced by George Matheron around 1960 under the name kriging.
- Their use in machine learning has grown substantially since the 1990s.
- A Gaussian process extends the multivariate Gaussian distribution to an infinite collection of random variables indexed by input points.
- Its most common use in machine learning is probabilistic non-linear regression.
- Gaussian processes are also applied in pattern classification, dimensionality reduction, missing-data, multi-task learning, and Bayesian optimization.

Gaussian process

For any finite set of points, this process defines a joint Gaussian:

$$p(\mathbf{f} \mid \mathbf{X}) = \mathcal{N}(\mathbf{f} \mid \boldsymbol{\mu}, \mathbf{K})$$

where $K_{ij} = k\left(\mathbf{x}_i, \mathbf{x}_j\right)$ and $\boldsymbol{\mu} = \left(m\left(\mathbf{x}_1\right), \ldots, m\left(\mathbf{x}_N\right)\right)$.

Gaussian Process (GP)

A Gaussian Process (GP) is denoted by:

$$f(\mathbf{x}) \sim GP(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

where $m(\mathbf{x})$ is the **mean function** and $k(\mathbf{x}, \mathbf{x}')$ is the **kernel** or **covariance function.**

Mean and covariance function

To fully specify the law of a GP, we need to specify mean and covariance functions:

$$f(\mathbf{x}_1), \dots, f(\mathbf{x}_N) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')), \qquad \forall\, \mathbf{x}, \mathbf{x}' \in \mathcal{X} \subset \mathbb{R}^d$$

where

mean function

$$m(\mathbf{x}) = \mathbb{E}\left[f(\mathbf{x})\right]$$

[kernel covariance function]

$$\begin{aligned} k\left(\mathbf{x}, \mathbf{x}'\right) &= \mathrm{Cov}\left[f(\mathbf{x}), f\left(\mathbf{x}'\right)\right] \\ &= \mathbb{E}\left[\left(f(\mathbf{x}) - m(\mathbf{x})\right)\left(f(\mathbf{x}') - m(\mathbf{x}')\right)\right] \end{aligned}$$

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}\left[f(\mathbf{x})\right]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^{\mathsf{T}}\mathbf{w}$
- Neural networks.

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}\left[f(\mathbf{x})\right]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$
- Neural networks.

Let $f(\mathbf{x}) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \alpha^{-1}\mathbf{I}\right)$.

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}[f(\mathbf{x})]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$
- Neural networks.

Let $f(\mathbf{x}) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$.

Then $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}[f(\mathbf{x})]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$
- Neural networks.

Let $f(\mathbf{x}) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$.

Then $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] = \mathbb{E}[\mathbf{w}^\mathsf{T}\phi(\mathbf{x})]$

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}[f(\mathbf{x})]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$
- Neural networks.

Let $f(\mathbf{x}) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$.

Then $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] = \mathbb{E}[\mathbf{w}^\mathsf{T}\phi(\mathbf{x})] = \mathbb{E}[\mathbf{w}^\mathsf{T}]\,\mathbb{E}[\phi(\mathbf{x})]$

Mean function $m(\mathbf{x})$

We can use any mean function we want $m(x) = \mathbb{E}[f(\mathbf{x})]$.

Popular choices are:

- $m(\mathbf{x}) = 0$
- $m(\mathbf{x}) = \text{const}$
- $m(x) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$
- Neural networks.

Let $f(\mathbf{x}) = \phi(\mathbf{x})^\mathsf{T}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$.

Then $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] = \mathbb{E}[\mathbf{w}^\mathsf{T}\phi(\mathbf{x})] = \mathbb{E}[\mathbf{w}^\mathsf{T}]\,\mathbb{E}[\phi(\mathbf{x})] = \mathbf{0}$

Kernel covariance function $k\left(\mathbf{x}, \mathbf{x}^{'}\right)$

The covariance function determines the nature of the GP, i.e., the hypothesis space/space of functions.

We usually use a covariance function that is a function of the indexes/locations:

$$k\left(\mathbf{x}, \mathbf{x}'\right) = \text{Cov}\left(f(\mathbf{x}), f\left(\mathbf{x}'\right)\right),$$

$k(\cdot, \cdot)$ must satisfy:

- Symmetry, i.e., $k\left(\mathbf{x}, \mathbf{x}'\right) = k\left(\mathbf{x}', \mathbf{x}\right)$.
- For any locations $\mathbf{x}_1, \ldots, \mathbf{x}_N$, the $N \times N$ Gram matrix $\mathbf{K}$ with $\mathbf{K}_{ij} = k\left(\mathbf{x}_i, \mathbf{x}_j\right)$ must be a positive semi-definite matrix.

Kernel covariance function $k\left(\mathbf{x}, \mathbf{x}^{'}\right)$

Let $f(\mathbf{x}) = \phi(\mathbf{x})^{\mathsf{T}}\mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \alpha^{-1}\mathbf{I}\right)$.

The kernel covariance $k(\mathbf{x}, \mathbf{x}')$ is given by:

$$\begin{aligned} k\left(\mathbf{x}, \mathbf{x}'\right) &= \mathbb{E}\left[f(\mathbf{x})f(\mathbf{x}')\right] \\ &= \phi(\mathbf{x})^{\mathsf{T}}\mathbb{E}\left[\mathbf{w}\mathbf{w}^{\mathsf{T}}\right]\phi(\mathbf{x}') \\ &= \phi(\mathbf{x})^{\mathsf{T}}\frac{\mathbf{I}}{\alpha}\phi(\mathbf{x}') \\ &= \frac{\phi(\mathbf{x})^{\mathsf{T}}\phi(\mathbf{x}')}{\alpha} \end{aligned}$$

Kernel covariance function $k\left(\mathbf{x}, \mathbf{x}^{'}\right)$

A widely used kernel in GPs is the squared-exponential or RBF (radial basis function) kernel is given by:

$$k\left(x, x'\right) = \sigma_f^2 \exp\left(-\frac{\left(x - x'\right)^2}{2\ell^2}\right), \qquad x \in \mathbb{R}$$

where σ_f^2 is the signal variance parameter and ℓ the length-scale parameter.

How do we draw samples from a GP?

- Given the mean function and covariance function for a GP, we can draw samples using a multivariate Gaussian distribution.
- To sample from the multivariate Gaussian distribution, we need a mean vector and a covariance matrix.
- The mean vector is obtained from the mean function.
- The covariance matrix is obtained from the covariance function.

Sampling from a GP, example

We have the set of x values:

$$\{x_1, x_2, \dots, x_N\} \subseteq \mathbb{R}$$

These are the indexes of the stochastic process.

We now compute the covariance matrix

$$\mathbf{K} = \begin{bmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \vdots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{bmatrix}$$

We assume the mean function is constant and equal to zero, i.e., $m(x) = 0 \,\forall\, x$.

Sampling from a GP, example

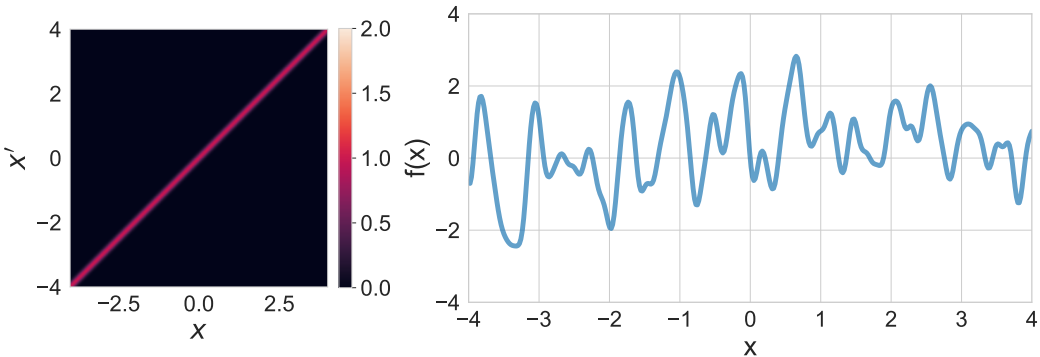
To generate functions from this GP, we will then sample from:

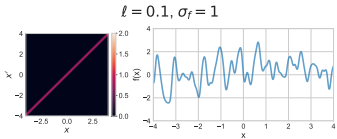
$$f(x_1), \dots, f(x_N) \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \vdots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{bmatrix} \right)$$

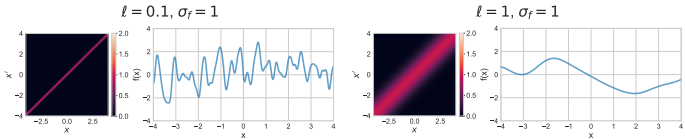
What we plot is x_i and $f(x_i)$, $\forall\, i$.

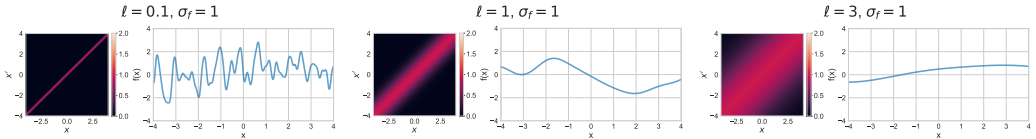
Sampling from a GP, example

$$\ell = 0.1,\ \sigma_f = 1$$

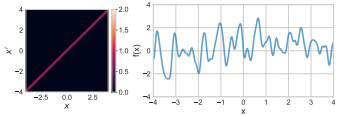



$\ell = 0.1, \sigma_f = 1$
x'
x
f(x)
x

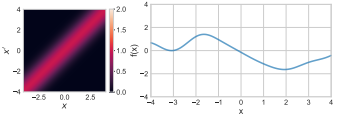

$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
x
x'
f(x)


$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
x
x'
f(x)

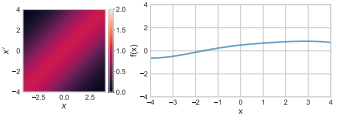
$\ell = 0.1, \sigma_f = 1$



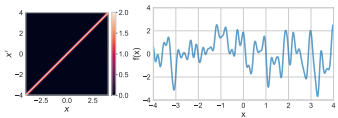
$\ell = 1, \sigma_f = 1$

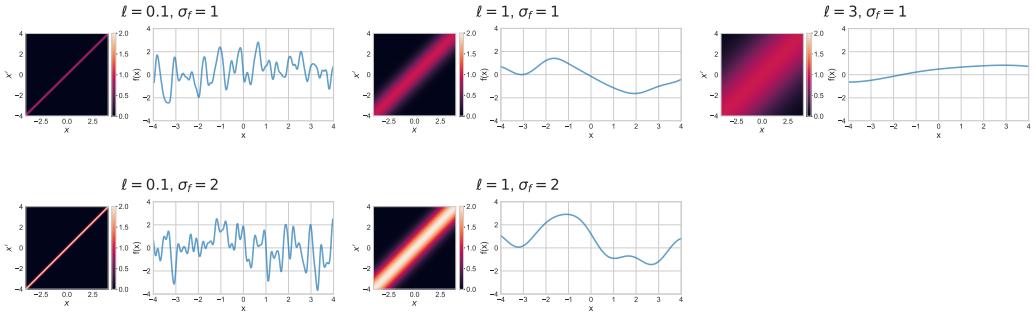


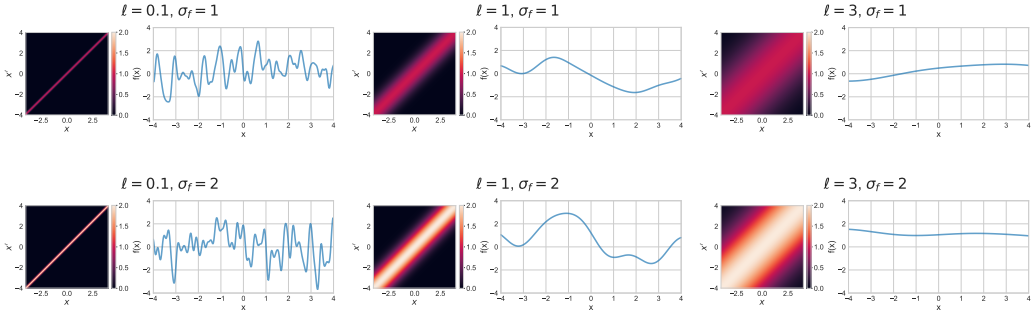
$\ell = 3, \sigma_f = 1$

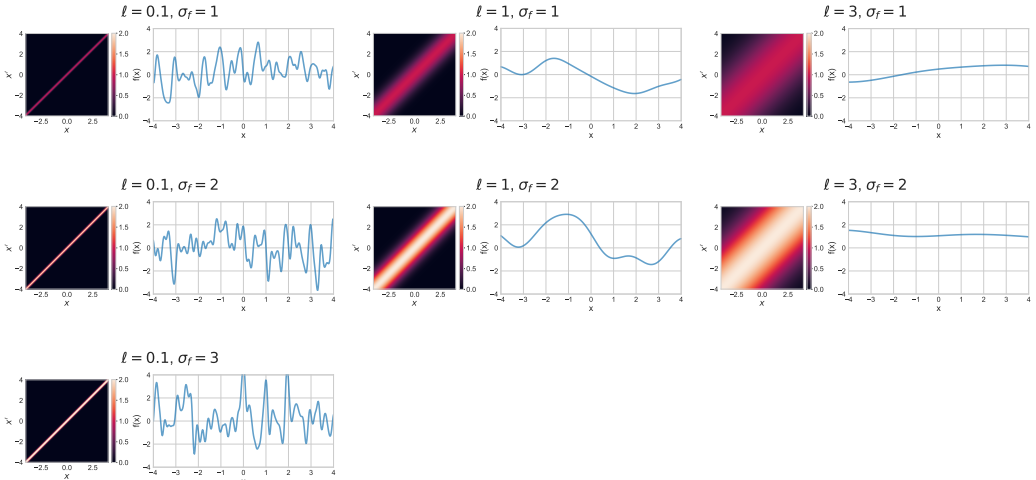


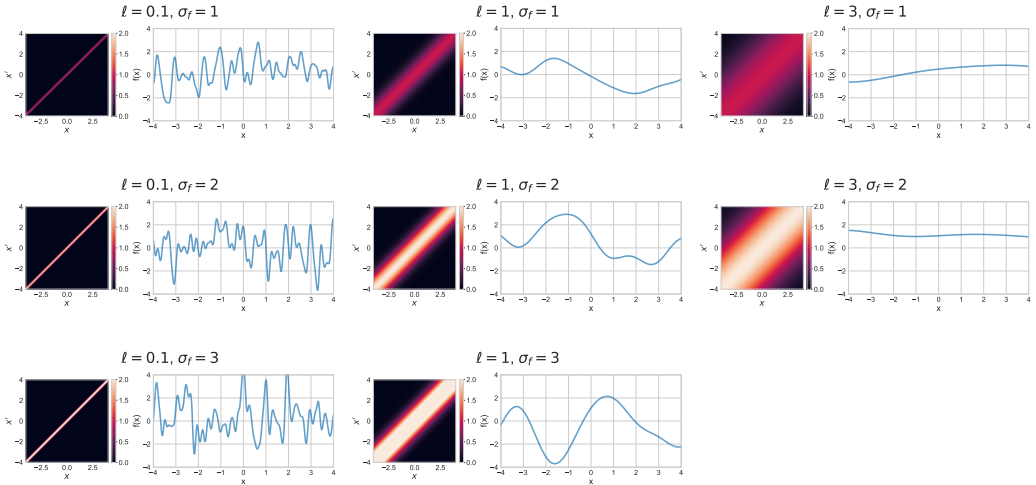
$\ell = 0.1, \sigma_f = 2$

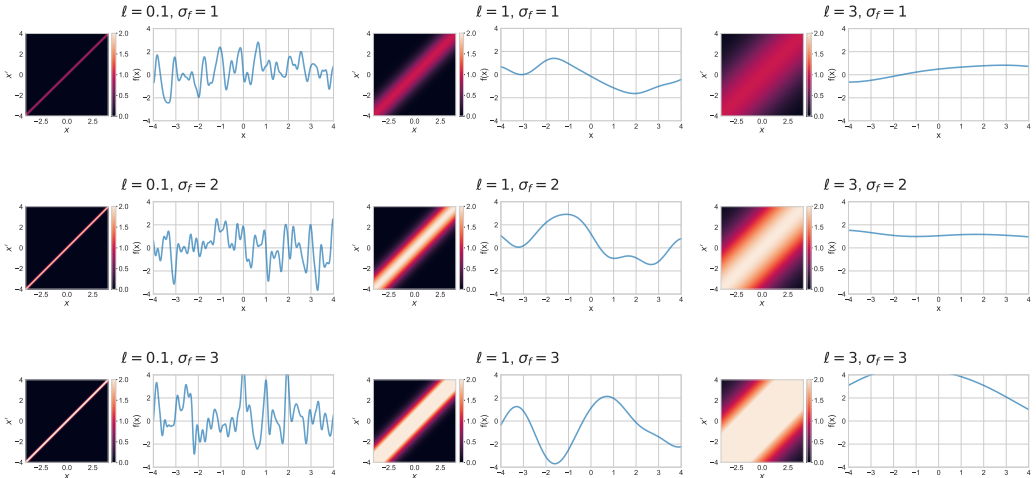


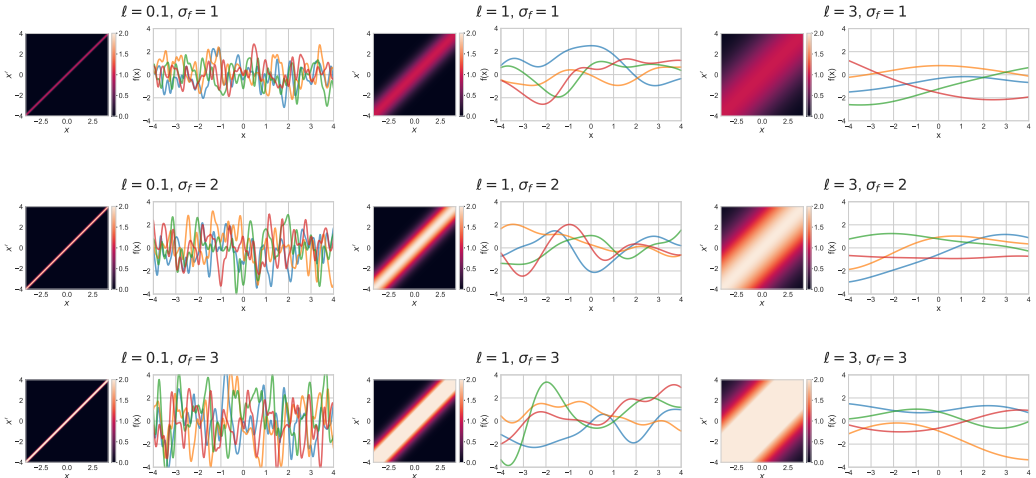

$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
x
x'
f(x)


$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
$\ell = 3, \sigma_f = 2$
x
x'
f(x)


$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
$\ell = 3, \sigma_f = 2$
$\ell = 0.1, \sigma_f = 3$
x
x'
f(x)

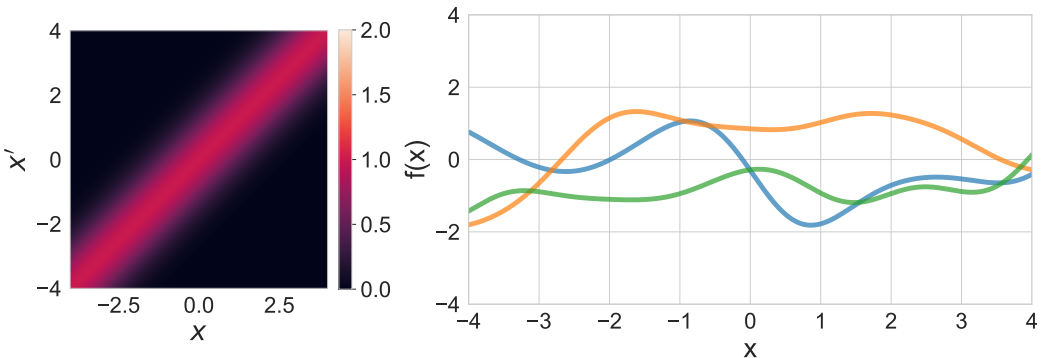

$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
$\ell = 3, \sigma_f = 2$
$\ell = 0.1, \sigma_f = 3$
$\ell = 1, \sigma_f = 3$
x
x'
f(x)


$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
$\ell = 3, \sigma_f = 2$
$\ell = 0.1, \sigma_f = 3$
$\ell = 1, \sigma_f = 3$
$\ell = 3, \sigma_f = 3$
x
x'
f(x)


$\ell = 0.1, \sigma_f = 1$
$\ell = 1, \sigma_f = 1$
$\ell = 3, \sigma_f = 1$
$\ell = 0.1, \sigma_f = 2$
$\ell = 1, \sigma_f = 2$
$\ell = 3, \sigma_f = 2$
$\ell = 0.1, \sigma_f = 3$
$\ell = 1, \sigma_f = 3$
$\ell = 3, \sigma_f = 3$
x
x'
f(x)

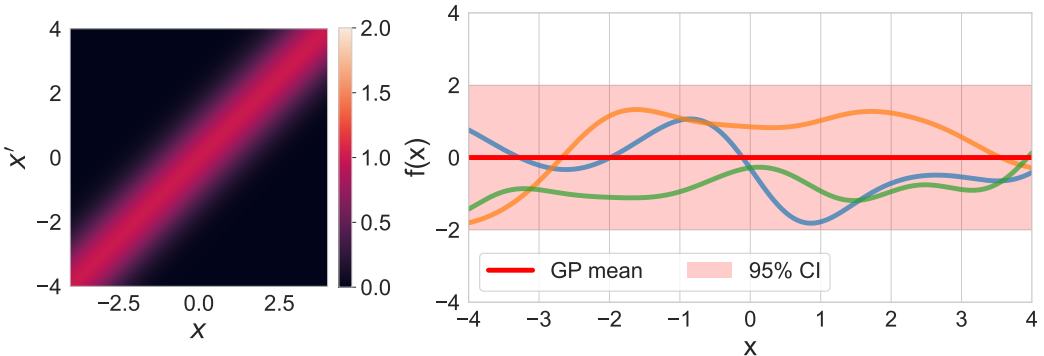
From prior to posterior

$$\ell = 1, \sigma_f = 1$$



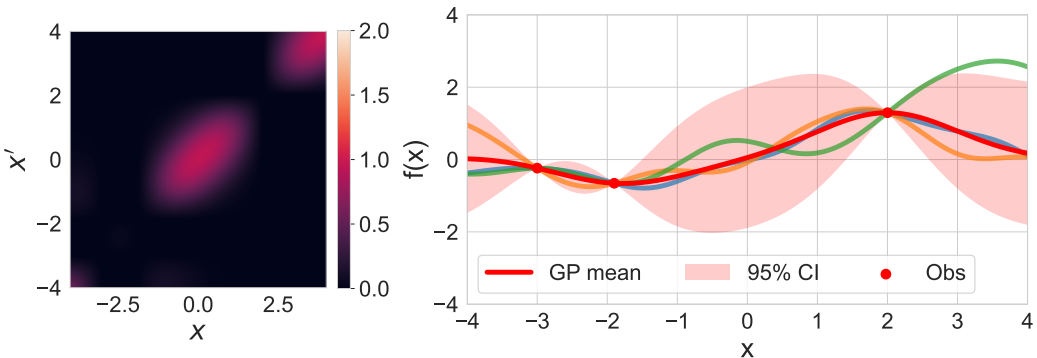
From prior to posterior

$$\ell = 1, \sigma_f = 1$$



From prior to posterior

$$\ell = 1, \sigma_f = 1$$



From prior to posterior

Training set:

$\mathcal{D} = \{(\mathbf{x}_i, f_i), i = 1 : N\}$

Test set:

$\{\mathbf{x}_i^*, i = 1 : N_*\}$

The training and test sets are organized as follows:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_N^\mathsf{T} \end{pmatrix}$$

training inputs

We want to predict the function outputs $\mathbf{f}^*$.

From prior to posterior

Training set:

$\mathcal{D} = \{(\mathbf{x}_i, f_i), i = 1 : N\}$

Test set:

$\{\mathbf{x}_i^*, i = 1 : N_*\}$

The training and test sets are organized as follows:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_N^\mathsf{T} \end{pmatrix} \qquad \mathbf{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_N \end{pmatrix}$$

training inputs **training targets**

We want to predict the function outputs $\mathbf{f}^*$.

From prior to posterior

Training set:

$\mathcal{D} = \{(\mathbf{x}_i, f_i), i = 1 : N\}$

Test set:

$\{\mathbf{x}_i^*, i = 1 : N_*\}$

The training and test sets are organized as follows:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_N^\mathsf{T} \end{pmatrix} \quad \mathbf{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_N \end{pmatrix} \quad \mathbf{X}^* = \begin{pmatrix} \mathbf{x}_1^{*\mathsf{T}} \\ \vdots \\ \mathbf{x}_{N_*}^{*\mathsf{T}} \end{pmatrix}$$

training inputs **training targets** **test inputs**

We want to predict the function outputs $\mathbf{f}^*$.

From prior to posterior

Training set:

$\mathcal{D} = \{(\mathbf{x}_i, f_i), i = 1 : N\}$

Test set:

$\{\mathbf{x}_i^*, i = 1 : N_*\}$

The training and test sets are organized as follows:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_N^\mathsf{T} \end{pmatrix} \quad \mathbf{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_N \end{pmatrix} \quad \mathbf{X}^* = \begin{pmatrix} \mathbf{x}_1^{*\mathsf{T}} \\ \vdots \\ \mathbf{x}_{N_*}^{*\mathsf{T}} \end{pmatrix} \quad \mathbf{f}^* = \begin{pmatrix} f_1^* \\ \vdots \\ f_{N_*}^* \end{pmatrix}$$

training inputs **training targets** **test inputs** **predictions**

We want to predict the function outputs $\mathbf{f}^*$.

From prior to posterior

The prior GP joint distribution of $\mathbf{f}$ and $\mathbf{f}^*$ is multivariate normal, i.e.:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^\mathsf{T} & \mathbf{K}_{**} \end{pmatrix} \right)$$

where:

$\mathbf{K}$ is of size $N \times N$, with $[\mathbf{K}]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

$\mathbf{K}_*$ is of size $N \times N_*$, with $[\mathbf{K}_*]_{ij} = k(\mathbf{x}_i, \mathbf{x}_{*j})$.

$\mathbf{K}_{**}$ is of size $N_* \times N_*$, with $[\mathbf{K}_{**}]_{ij} = k(\mathbf{x}_{*i}, \mathbf{x}_{*j})$.

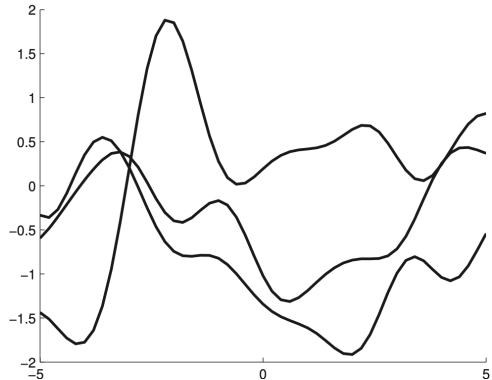
From prior to posterior

Conditioning on the training targets $\mathbf{f}$ gives the posterior GP, i.e.:

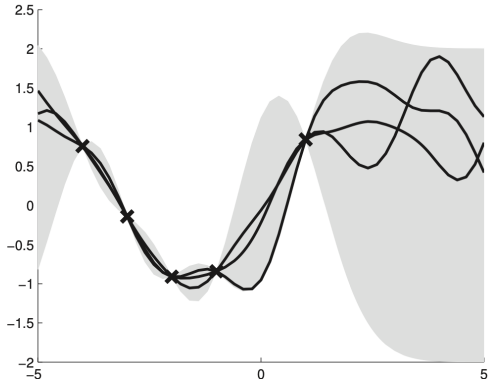
$$\mathbf{f}_* \mid \mathbf{X}_*, \mathbf{X}, \mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}_*, \Sigma_*)$$

The posterior has the following form:

$$\boldsymbol{\mu}_* = \boldsymbol{\mu}(\mathbf{X}_*) + \mathbf{K}_*^\intercal \mathbf{K}^{-1}(\mathbf{f} - \boldsymbol{\mu})$$
$$\boldsymbol{\Sigma}_* = \mathbf{K}_{**} - \mathbf{K}_*^\intercal \mathbf{K}^{-1} \mathbf{K}_*$$



(a)

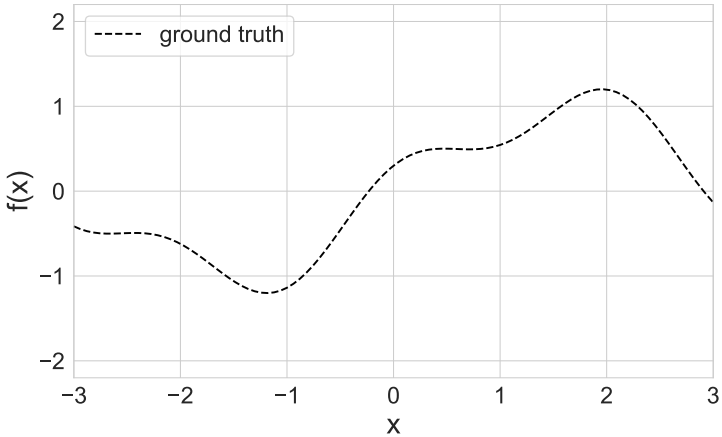


(b)

(a) Samples from the prior, $p(\mathbf{f} \mid \mathbf{X})$, using a squared exponential kernel.

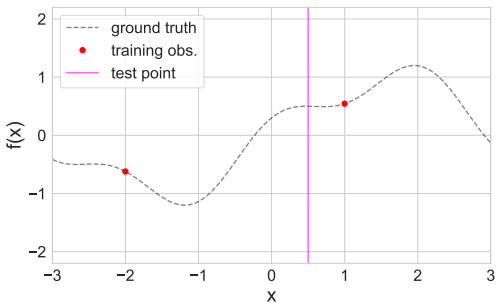
(b) Samples from a GP posterior, $p\left(\mathbf{f}_* \mid \mathbf{X}_*, \mathbf{X}, \mathbf{f}\right)$.

Numerical example



Numerical example

Numerical example



Training samples:

$$(x_1, f(x_1)) = (-2, -0.62)$$
$$(x_2, f(x_2)) = (1, 0.54)$$

$$x^* = 0.5$$
$$f^* = ?$$

Numerical example

Suppose f is a Gaussian process, then

$$\underbrace{f(x_1), f(x_2)}_{\text{training samples}}, \underbrace{f(x^*)}_{\text{test sample}} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$$

where:

$$\boldsymbol{\Sigma} = \left(\begin{array}{cc|c} k(x_1, x_1) & k(x_1, x_2) & k(x_1, x^*) \\ k(x_2, x_1) & k(x_2, x_2) & k(x_2, x^*) \\ \hline k(x^*, x_1) & k(x^*, x_2) & k(x^*, x^*) \end{array}\right) = \left(\begin{array}{cc|c} 1 & 0.01 & 0.04 \\ 0.01 & 1 & 0.88 \\ \hline 0.04 & 0.88 & 1 \end{array}\right)$$

The value of the function f^* at the testing point $x^* = 0.5$ is computed from $p(\mathbf{f}_* \mid \mathbf{f}) = \mathcal{N}(\boldsymbol{\mu}_*, {}_*)$ with:

$$\begin{aligned} \boldsymbol{\mu}_* &= \mathbf{K}_*^\intercal \mathbf{K}^{-1} (\mathbf{f} - \boldsymbol{\mu}) \\ &= \begin{pmatrix} k(x^*, x_1) & k(x^*, x_2) \end{pmatrix} \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) \\ k(x_2, x_1) & k(x_2, x_2) \end{pmatrix}^{-1} \begin{pmatrix} f(x_1) \\ f(x_2) \end{pmatrix} \\ &= 0.46 \end{aligned}$$

Numerical example

Suppose f is a Gaussian process, then

$$\underbrace{f(x_1), f(x_2)}_{\text{training samples}}, \underbrace{f(x^*)}_{\text{test sample}} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$$

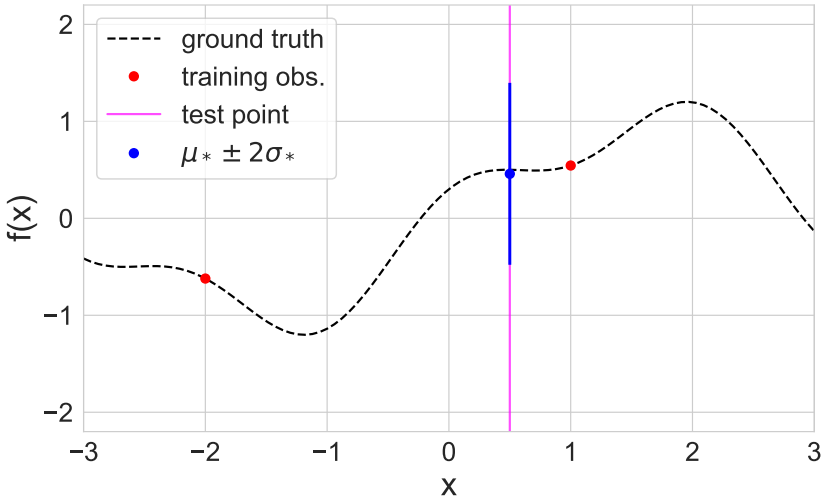
where:

$$\boldsymbol{\Sigma} = \left(\begin{array}{cc|c} k(x_1, x_1) & k(x_1, x_2) & k(x_1, x^*) \\ k(x_2, x_1) & k(x_2, x_2) & k(x_2, x^*) \\ \hline k(x^*, x_1) & k(x^*, x_2) & k(x^*, x^*) \end{array}\right) = \left(\begin{array}{cc|c} 1 & 0.01 & 0.04 \\ 0.01 & 1 & 0.88 \\ \hline 0.04 & 0.88 & 1 \end{array}\right)$$

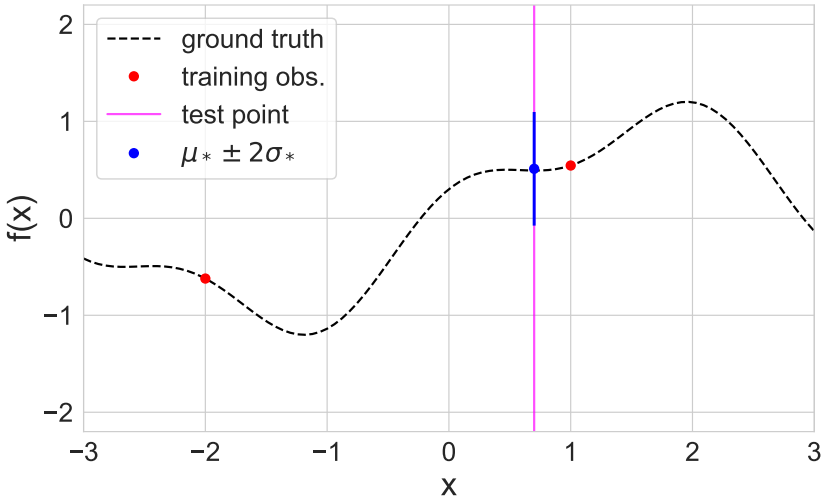
The value of the function f^* at the testing point $x^* = 0.5$ is computed from $p(\mathbf{f}_* \mid \mathbf{f}) = \mathcal{N}(\boldsymbol{\mu}_*, {}_*)$ with:

$$\begin{aligned} \boldsymbol{\Sigma}_* &= \mathbf{K}_{**} - \mathbf{K}_*^\intercal \mathbf{K}^{-1} \mathbf{K}_* \\ &= k(x^*, x^*) - \begin{pmatrix} k(x^*, x_1) & k(x^*, x_2) \end{pmatrix} \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) \\ k(x_2, x_1) & k(x_2, x_2) \end{pmatrix}^{-1} \begin{pmatrix} k(x^*, x_1) \\ k(x^*, x_2) \end{pmatrix} \\ &= 0.22 \end{aligned}$$

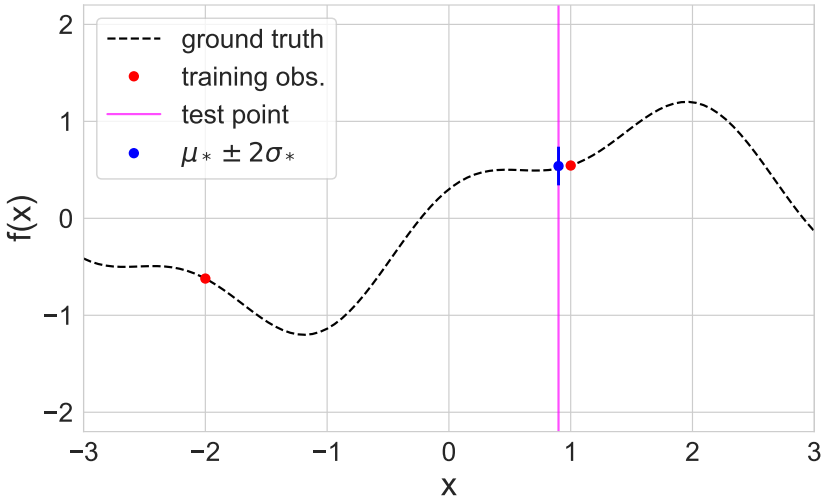
Numerical example



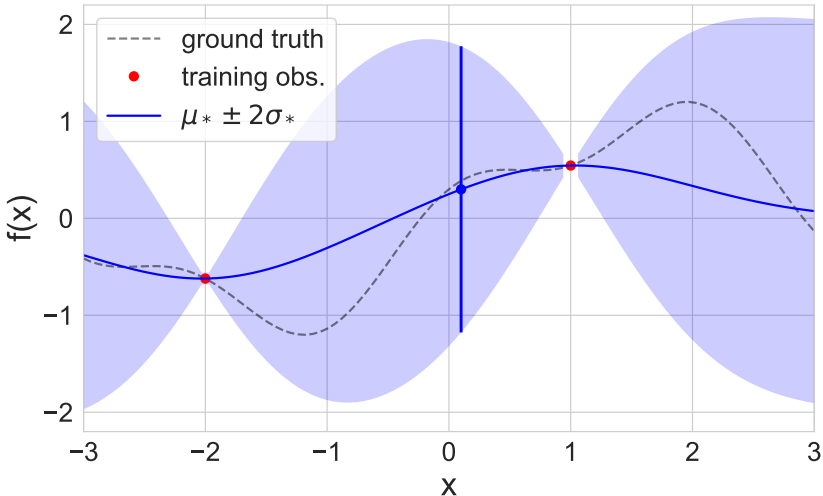
Numerical example



Numerical example



Numerical example



Predictions using noisy observations

Noisy version of the underlying function:

$$y = f(\mathbf{x}) + \epsilon, \qquad \epsilon \sim \mathcal{N}\left(0, \sigma_y^2\right)$$

The covariance of the observed noisy responses is:

$$\text{cov}\left[y_p, y_q\right] = K\left(\mathbf{x}_p, \mathbf{x}_q\right) + \sigma_y^2 \mathbb{I}(p = q)$$

$$\text{cov}[\mathbf{y} \mid \mathbf{X}] = \mathbf{K} + \sigma_y^2 \mathbf{I}_N \triangleq \mathbf{K}_y$$

Predictions using noisy observations

The joint density of training and test samples is given by:

$$y_1, y_2, \dots, y_N, f_* \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \mathbf{K}_y & \mathbf{k}_* \\ \mathbf{k}_*^\mathsf{T} & k_{**} \end{pmatrix}\right)$$

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \mathbf{K}_y & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{pmatrix}\right)$$

Predictions using noisy observations

The posterior predictive density is:

$$
\begin{aligned}
p\left(\mathbf{f}_* \mid \mathbf{X}_*, \mathbf{X}, \mathbf{y}\right) &= \mathcal{N}\left(\mathbf{f}_* \mid \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*\right) \\
\boldsymbol{\mu}_* &= \mathbf{K}_*^T \mathbf{K}_y^{-1} \mathbf{y} \\
\boldsymbol{\Sigma}_* &= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}_y^{-1} \mathbf{K}_*
\end{aligned}
$$

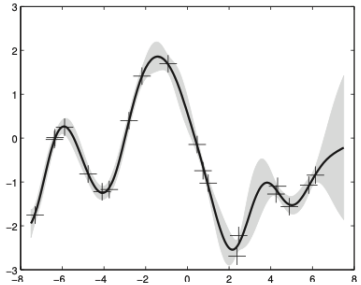
In the case of a single test input, this simplifies as follows:

$$p\left(f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}\right) = \mathcal{N}\left(f_* \mid \mathbf{k}_*^T \mathbf{K}_y^{-1} \mathbf{y}, k_{**} - \mathbf{k}_*^T \mathbf{K}_y^{-1} \mathbf{k}_*\right)$$

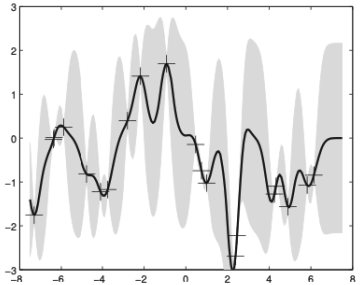
where $\mathbf{k}_* = \left[\kappa\left(\mathbf{x}_*, \mathbf{x}_1\right), \ldots, \kappa\left(\mathbf{x}_*, \mathbf{x}_N\right)\right]$ and $k_{**} = \kappa\left(\mathbf{x}_*, \mathbf{x}_*\right)$. Another way to write the posterior mean is as follows:

$$\bar{f}_* = \mathbf{k}_*^T \mathbf{K}_y^{-1} \mathbf{y} = \sum_{i=1}^{N} \alpha_i \kappa\left(\mathbf{x}_i, \mathbf{x}_*\right)$$

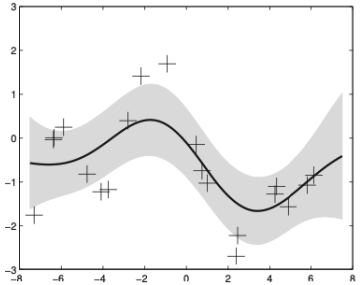
where $\boldsymbol{\alpha} = \mathbf{K}_y^{-1} \mathbf{y}$.



(a)



(b)



(c)

Which kernel functions are appropriate for a given problem?

How does the choice of different hyperparameter values affect the resulting model?

Kernels in Gaussian Processes

A **kernel** (or covariance function) $k(\mathbf{x}, \mathbf{x}')$ is the foundation of a Gaussian process. It defines the covariance structure between function values at any two input points, completely specifying the GP's prior distribution and determining:

- **Smoothness** of sample functions
- **Length-scale** (characteristic distance over which correlations decay)
- **Amplitude** (overall vertical scale of variations)
- **Periodicity** (if applicable)
- **Non-stationarity** (if applicable)

Kernels must be **positive semi-definite** to ensure valid covariance matrices.

Kernel Effects

1. **Signal Variance (σ_f^2)**
   - Controls the **amplitude** or overall vertical scale of function variations.
   - Larger values: function can vary over a wider range.
   - Smaller values: function values stay closer to the mean.
2. **Length-Scale (ℓ)**
   - Controls how quickly correlations **decay with distance.**
   - **Large** ℓ: Points remain strongly correlated over longer distances. Sample functions are smooth and slowly varying.
   - **Small** ℓ: Correlations decay rapidly, function can vary more rapidly and rougher behavior.

Two common kernels

RBF (squared exponential) kernel:

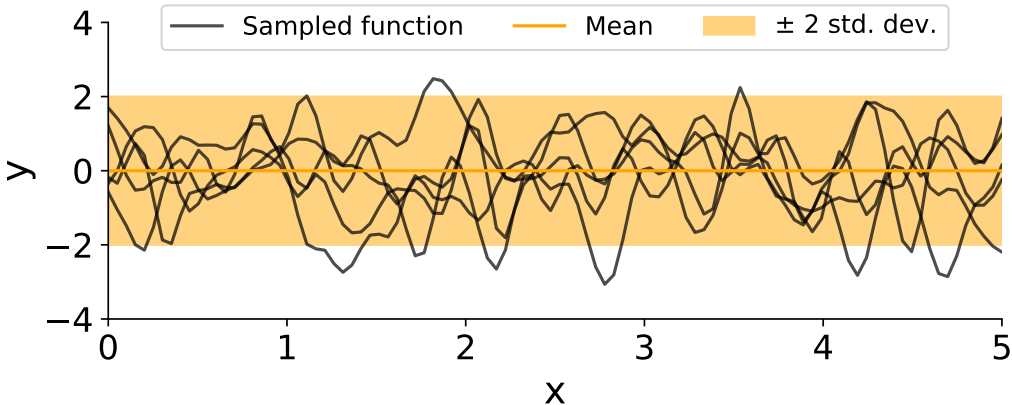
$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') \;=\; \sigma_f^2 \,\exp\!\left(-\frac{\|\mathbf{x}-\mathbf{x}'\|^2}{2\,\ell^2}\right).$$

Matérn kernel (general $\nu > 0$) can be written as:

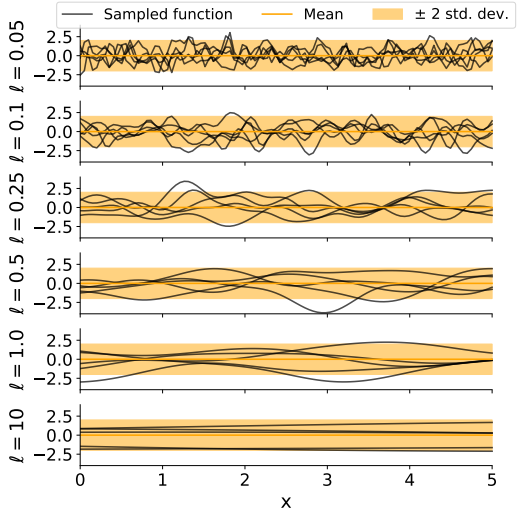
$$k_{\text{Matérn}}(\mathbf{x}, \mathbf{x}') \;=\; \sigma_f^2 \,\frac{1}{\Gamma(\nu)\,2^{\nu-1}} \left(\frac{\sqrt{2\nu}}{\ell}\,\|\mathbf{x}-\mathbf{x}'\|\right)^{\nu} K_\nu\!\left(\frac{\sqrt{2\nu}}{\ell}\,\|\mathbf{x}-\mathbf{x}'\|\right),$$

where K_ν is the modified Bessel function of the second kind, ν controls smoothness, ℓ is the length scale, and σ_f^2 is the signal variance.

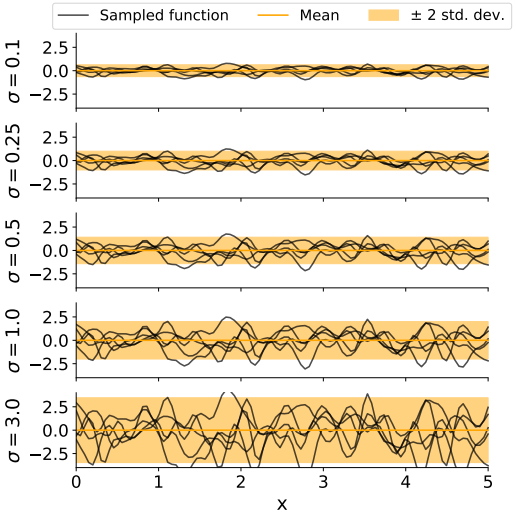
RBF kernel - samples from prior



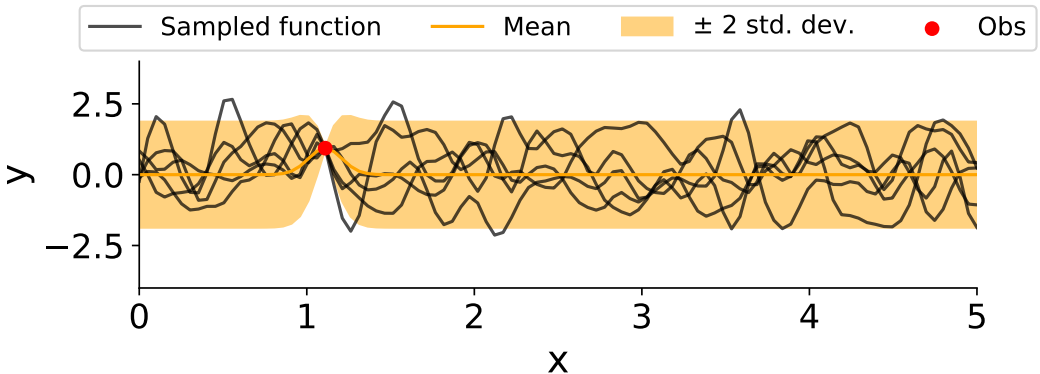
RBF kernel - samples from prior



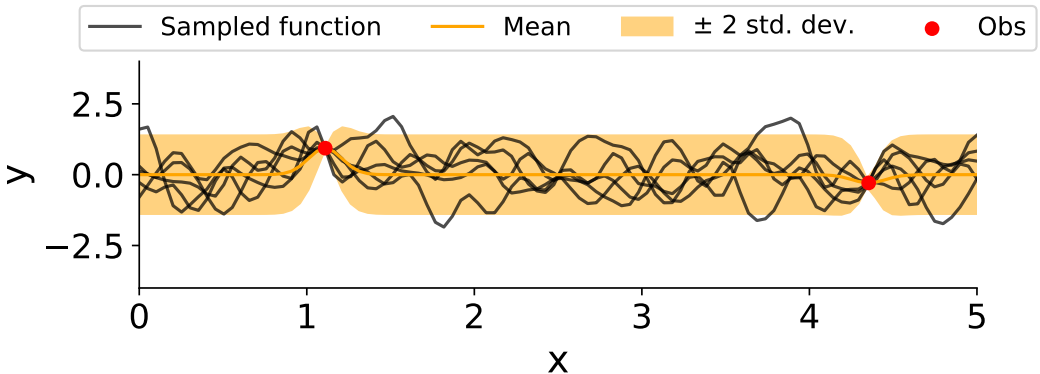
RBF kernel - samples from prior



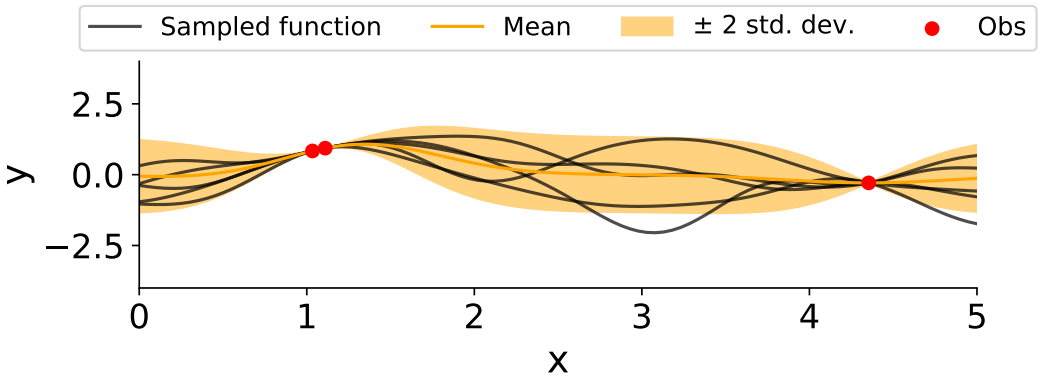
RBF kernel - samples from posterior



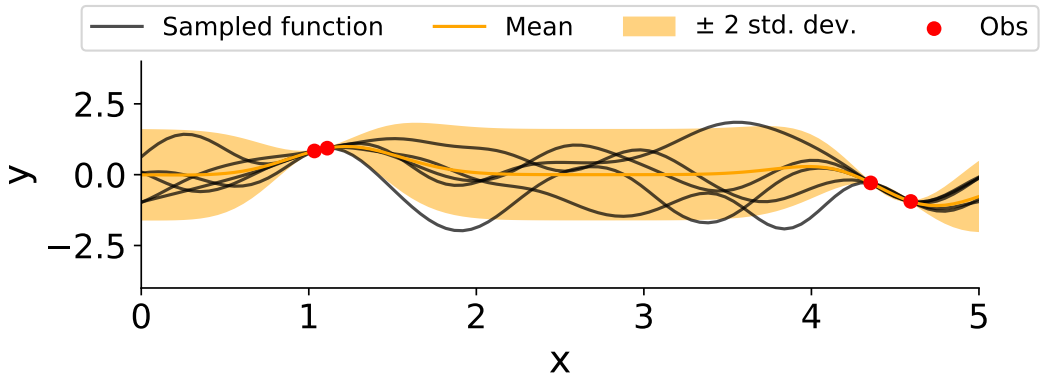
RBF kernel - samples from posterior



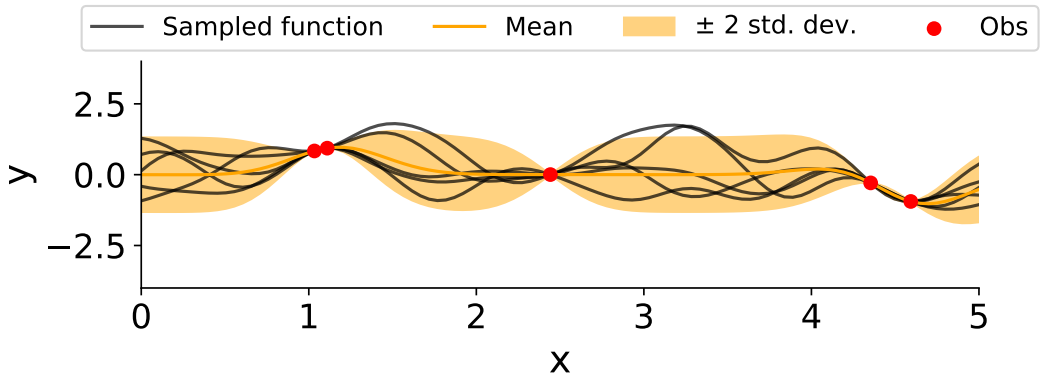
RBF kernel - samples from posterior



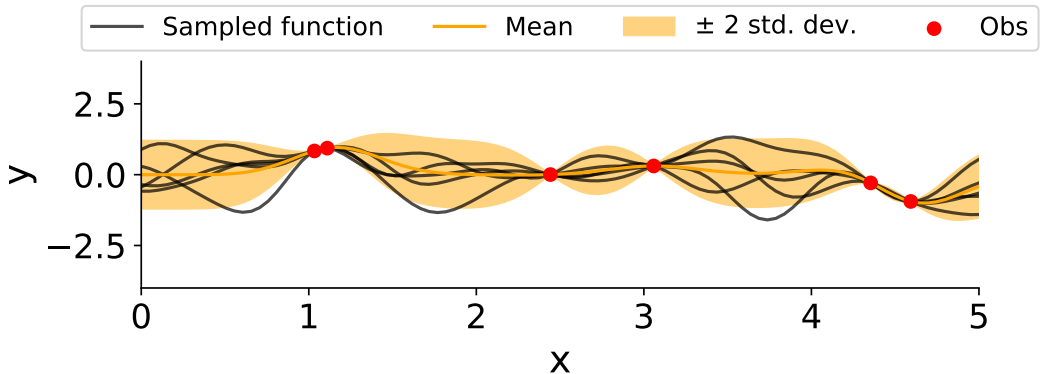
RBF kernel - samples from posterior



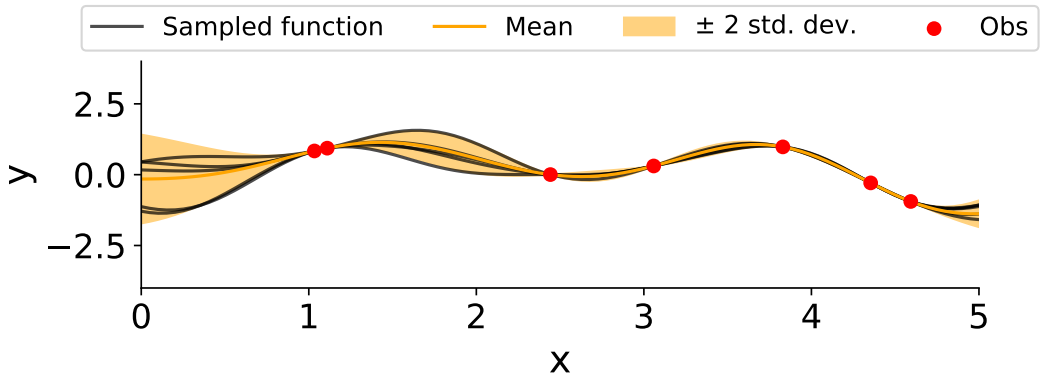
RBF kernel - samples from posterior



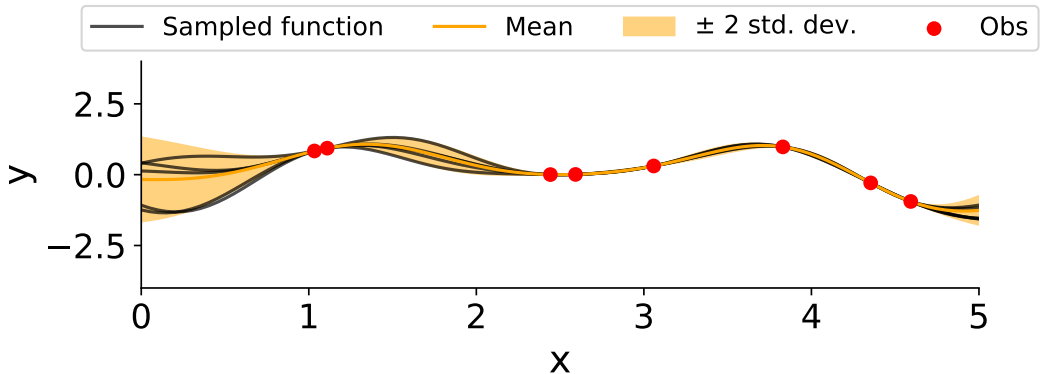
RBF kernel - samples from posterior



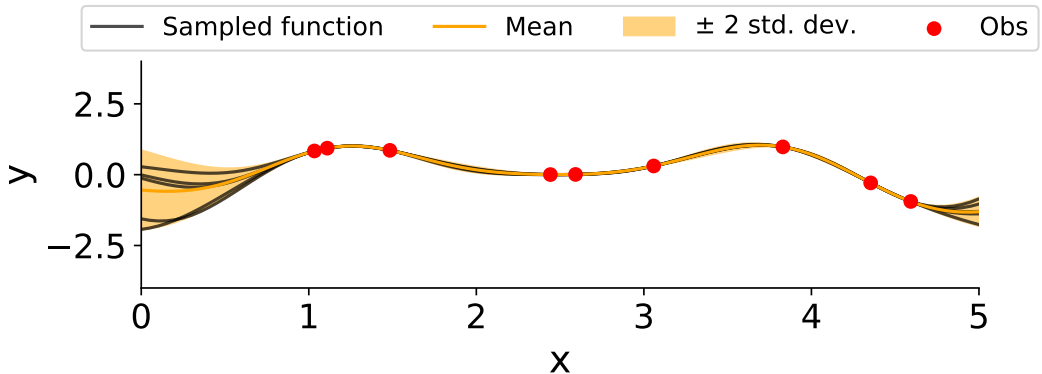
RBF kernel - samples from posterior



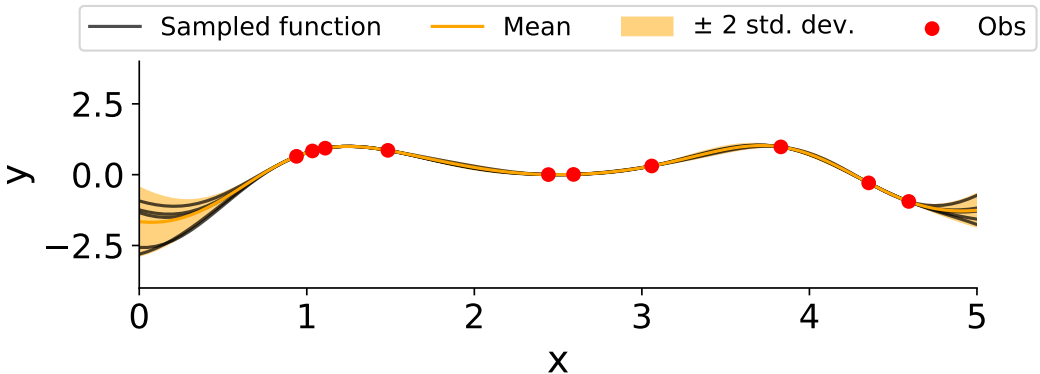
RBF kernel - samples from posterior



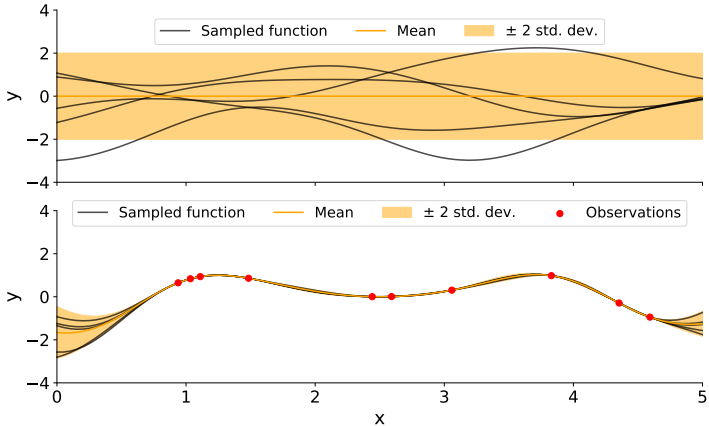
RBF kernel - samples from posterior



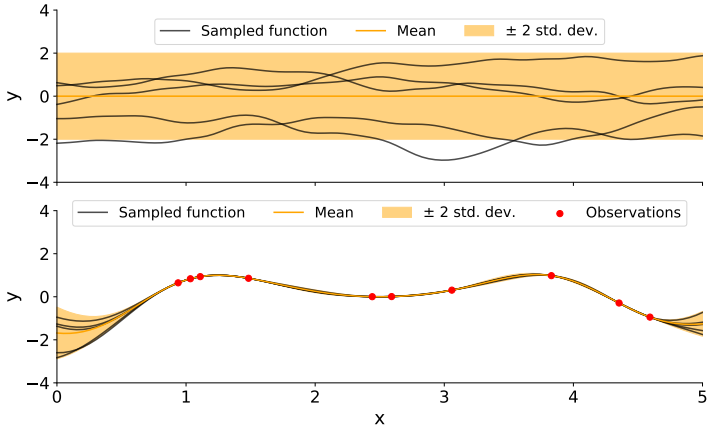
RBF kernel - samples from posterior



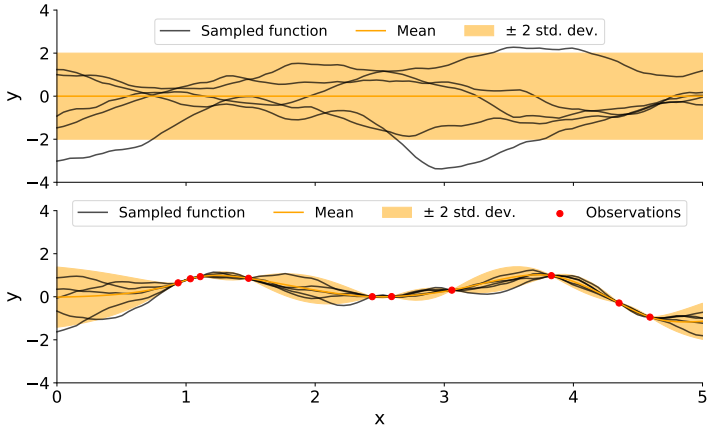
RBF kernel - prior / posterior



Rational-quadratic kernel - prior / posterior



Matern kernel - prior / posterior



3. **Stationarity**

- **Stationary kernel**: Covariance depends only on the *difference* $\mathbf{x} - \mathbf{x}'$, invariant under translation
- **Non-stationary kernel**: Covariance depends on absolute locations, allowing position-dependent behavior

4. **Isotropy**

- **Isotropic kernel**: Covariance depends only on the *Euclidean distance* $\|\mathbf{x} - \mathbf{x}'\|$
- **Anisotropic kernel**: Different characteristic length-scales and couplings across dimensions

Stationary Kernels

A stationary covariance function satisfies:

$$\text{Cov}[f(\mathbf{x}), f(\mathbf{x}')] = k(\mathbf{x} - \mathbf{x}') = \psi(\mathbf{x} - \mathbf{x}')$$

Translation invariant: $k(\mathbf{x} + \mathbf{a}, \mathbf{x}' + \mathbf{a}) = k(\mathbf{x}, \mathbf{x}')$ for all shifts $\mathbf{a}$.

Example: RBF kernel

Isotropic Kernels

An isotropic kernel depends only on distance:

$$k(\mathbf{x}, \mathbf{x}') = \phi(\|\mathbf{x} - \mathbf{x}'\|)$$

where $\phi : [0, \infty) \to \mathbb{R}$ is a scalar function.

Property: Every isotropic kernel is stationary, but not all stationary kernels are isotropic.

Example: RBF kernel is isotropic when $\Lambda = \ell^2 I$ (identity matrix)

Anisotropic Kernels

A stationary *but not isotropic* kernel depends on the full lag vector $\mathbf{h} = \mathbf{x} - \mathbf{x}'$, including directional information:

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}')^\intercal \Lambda^{-1} (\mathbf{x} - \mathbf{x}')\right)$$

where Λ is a positive definite matrix defining different characteristic length-scales and couplings across dimensions.
Use case: When different input dimensions have different "importance" or correlation decay rates.

Non-Stationary Kernels

Covariance depends on absolute locations, not just their difference. Allow position-dependent behavior and varying smoothness.

Examples: Polynomial kernel, neural network kernel

Kernel: Squared-Exponential (RBF)

1D Form:

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

Multi-dimensional isotropic form:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$$

Characteristics: Infinitely differentiable (very smooth sample functions), **stationary and isotropic**, rapid decay with distance, widely used due to simplicity and smoothness.

Parameters: σ_f^2, signal variance (amplitude); ℓ, length-scale.

Kernel: Matérn

General form:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{x}'\|}{\ell} \right)^{\nu} K_\nu \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{x}'\|}{\ell} \right)$$

where K_ν is a modified Bessel function, $\nu > 0$ controls smoothness, and ℓ is the length-scale.

Common versions: $\nu = 1/2$, exponential kernel; $\nu = 3/2$; $\nu \to \infty$, approaches RBF kernel.

Characteristics: More flexible smoothness control than RBF, includes exponential kernel as special case, stationary and isotropic.

Parameters: σ_f^2, signal variance; ℓ, length-scale; ν, smoothness parameter.

Kernel: Rational Quadratic (RQ)

Form:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\alpha\ell^2}\right)^{-\alpha}$$

Characteristics: Mixture of RBF kernels with different length-scales; can be viewed as weighted superposition of exponential kernels at different scales, **stationary and isotropic.**

Parameters: σ_f^2, signal variance, ℓ, length-scale; α, mixing parameter (larger α, closer to RBF).

Kernel: Periodic

Form:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{2\sin^2(\pi|\mathbf{x} - \mathbf{x}'|/p)}{\ell^2}\right)$$

Characteristics: Captures periodic patterns in data; essential for time-series with seasonal patterns.

Parameters: σ_f^2: signal variance; ℓ: length-scale; p: period (wavelength).

Kernel: Polynomial

Form:

$$k(\mathbf{x}, \mathbf{x}') = \left(\sigma_f^2 \mathbf{x} \cdot \mathbf{x}' + c\right)^d$$

or

$$k(\mathbf{x}, \mathbf{x}') = \left(\sigma_f^2 \mathbf{x} \cdot \mathbf{x}' + c\right)^d + \sigma_n^2$$

Characteristics: Often used with small d (typically 1, 2, or 3), **non-stationary kernel.**

Parameters: σ_f^2, signal variance, c, offset, d, degree (model complexity), σ_n^2, noise variance (if included).

Kernel: White Noise

Form:

$$k(x, x') = \sigma_n^2 \delta(x - x')$$

where $\delta(\cdot)$ is the Dirac delta function.

In practice (discrete inputs with finite precision):

$$k_{\text{white noise}}(\mathbf{x}, \mathbf{x}') = \begin{cases} \sigma_n^2 & \text{if } \mathbf{x} = \mathbf{x}' \\ 0 & \text{otherwise} \end{cases}$$

Kernel: White Noise

- **Pure noise**: No correlation between any two distinct points.
- **Variance only on diagonal**: Contributes σ_n^2 to the diagonal of the covariance matrix.
- **Numerical stability**: Often added to smooth kernels to stabilize matrix inversion.
- **Independent observations**: Assumes each observation contains independent measurement noise.

Kernel: White Noise

1. **Likelihood noise term**: Observations $y_i = f(\mathbf{x}_i) + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$

2. **Jitter for numerical stability**: Added to other kernels to improve matrix conditioning:

$$k_{\text{total}} = k_{\text{smooth}}(\mathbf{x}, \mathbf{x}') + \sigma_n^2 \delta(\mathbf{x} - \mathbf{x}')$$

3. **Model white noise in data**: When you expect uncorrelated noise in observations

Kernel combinations

Kernels are closed under addition and multiplication:

- **Sum (additive)**: $k_{\text{total}} = k_1 + k_2$ (combines features)
  - Example: RBF + Periodic (smooth + periodic patterns)
  - Example: RBF + White Noise (smooth signal + independent noise)
- **Product (multiplicative)**: $k_{\text{total}} = k_1 \times k_2$ (modulates one kernel by another)
  - Example: (RBF) $\times$ (Periodic) (smooth periodic trends)

Estimating the kernel parameters

$$K_y(x_p, x_q) = {\sigma_f}^2 \exp\left(-\frac{1}{2\ell^2}(x_p - x_q)^2\right) + {\sigma_y}^2 \delta_{pq}$$

We can maximize the marginal likelihood:

$$p(\mathbf{y} \mid \mathbf{X}) = \int p(\mathbf{y} \mid \mathbf{f}, \mathbf{X}) p(\mathbf{f} \mid \mathbf{X}) d\mathbf{f}$$

Estimating the kernel parameters

We can maximize the marginal likelihood:

$$p(\mathbf{y} \mid \mathbf{X}) = \int p(\mathbf{y} \mid \mathbf{f}, \mathbf{X}) p(\mathbf{f} \mid \mathbf{X}) d\mathbf{f}$$

where:

$$p(\mathbf{f} \mid \mathbf{X}) = \mathcal{N}(\mathbf{f} \mid \mathbf{0}, \mathbf{K})$$

$$p(\mathbf{y} \mid \mathbf{f}) = \prod_i \mathcal{N}\left(y_i \mid f_i, \sigma_y^2\right)$$

Estimating the kernel parameters

The marginal likelihood is given by:

$$\begin{aligned}\log p(\mathbf{y} \mid \mathbf{X}) &= \log \mathcal{N}\left(\mathbf{y} \mid \mathbf{0}, \mathbf{K}_y\right) \\ &= \underbrace{-\frac{1}{2}\mathbf{y}\mathbf{K}_y^{-1}\mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2}\log\left|\mathbf{K}_y\right|}_{\text{model complexity}} \underbrace{-\frac{N}{2}\log(2\pi)}_{\text{constant}}\end{aligned}$$

Estimating the kernel parameters

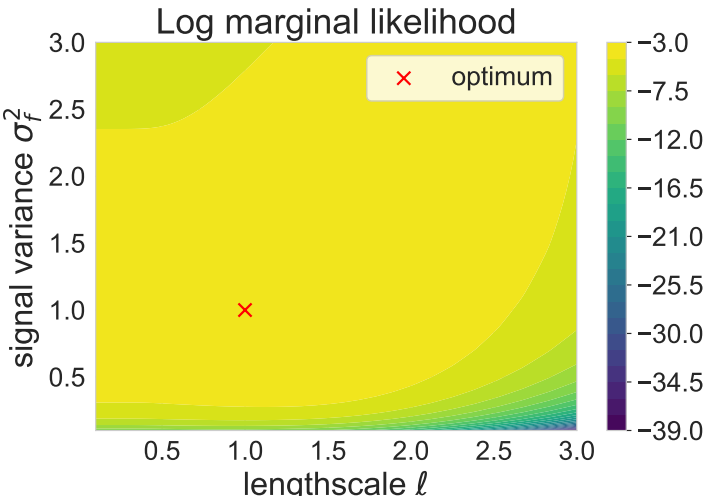
Let the kernel parameters (also called hyper-parameters) be denoted by $\boldsymbol{\theta}$.

One can show that:

$$\begin{aligned}\frac{\partial}{\partial \theta_j} \log p(\mathbf{y} \mid \mathbf{X}) &= \frac{1}{2}\mathbf{y}^T \mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \mathbf{K}_y^{-1}\mathbf{y} - \frac{1}{2}\,\mathsf{tr}\left(\mathbf{K}_y^{-1}\frac{\partial \mathbf{K}_y}{\partial \theta_j}\right) \\ &= \frac{1}{2}\,\mathsf{tr}\left(\left(\boldsymbol{\alpha}\boldsymbol{\alpha}^T - \mathbf{K}_y^{-1}\right)\frac{\partial \mathbf{K}_y}{\partial \theta_j}\right)\end{aligned}$$

where $\boldsymbol{\alpha} = \mathbf{K}_y^{-1}\mathbf{y}$.

Log marginal likelihood



Computational cost

- One difficulty with GPs is the computational cost of training them: $O\left(n^3\right)$ (and $O\left(n^2\right)$ memory).
- They work our of the box for n in the order of a few thousands.
- There are many ways to side-step this cost: inducing inputs, efficient matrix-vector multiplications, random features, etc.
- These days we can use GPs for n in the order of tens of millions.

Applications

Automatic Relevance Determination (ARD)

Multi-task Gaussian processes

Gaussian processes and neural networks

Deconvolution using Gaussian processes

Automatic Relevance Determination (ARD)

- **Automatic Relevance Determination (ARD)** is a technique for determining the importance of input variables.
- Originally formulated in **neural networks** (MacKay 1994, Neal 1996).
- Extensively used in **Gaussian Process** regression.
- Optimize separate length-scale parameters for each input dimension.

ARD Kernel Function

Standard squared-exponential kernel with ARD:

$$k(x_n, x_m) = \sigma_0 \exp\left(-\frac{1}{2}\sum_{i=1}^{D}\lambda_i(x_{ni} - x_{mi})^2\right) + \sigma_3 x_n^T x_m$$

- σ_0: overall variance
- λ_i: **precision for dimension** i
- σ_3: linear term

- If λ_i is **large**, dimension i is **important**
- If λ_i is **small**, dimension i is **irrelevant**

ARD Kernel Function

Large λ_i (Relevant)

Steep kernel in dimension i
Function changes rapidly with x_i
Strong dependence on input i

Small λ_i (Irrelevant)

Flat kernel in dimension i
Function changes slowly with x_i
Weak/no dependence on input i

Example: $\lambda_1 = 1$ vs $\lambda_2 = 0.01$

- Function is sensitive to x_1.
- Function is nearly insensitive to x_2.

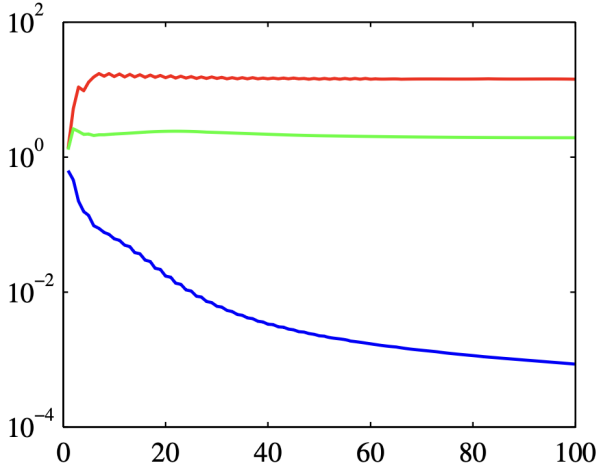
ARD

Synthetic Dataset:

- x_1: generated from Gaussian.
- **Target**: $t = \sin(2x_1) +$ Gaussian noise.
- x_2: copy of x_1 + noise (partially correlated).
- x_3: independent Gaussian (irrelevant).

Example Results

λ_1 (red), λ_2 (green), λ_3 (blue)



General Form with Linear Terms

Extended ARD kernel (for regression):

$$k(x_n, x_m) = \sigma_0 \exp\left(-\frac{1}{2}\sum_{i=1}^{D}\lambda_i(x_{ni} - x_{mi})^2\right) + \sigma_3 \sum_{i=1}^{D} x_{ni}x_{mi}$$

Two competing mechanisms:

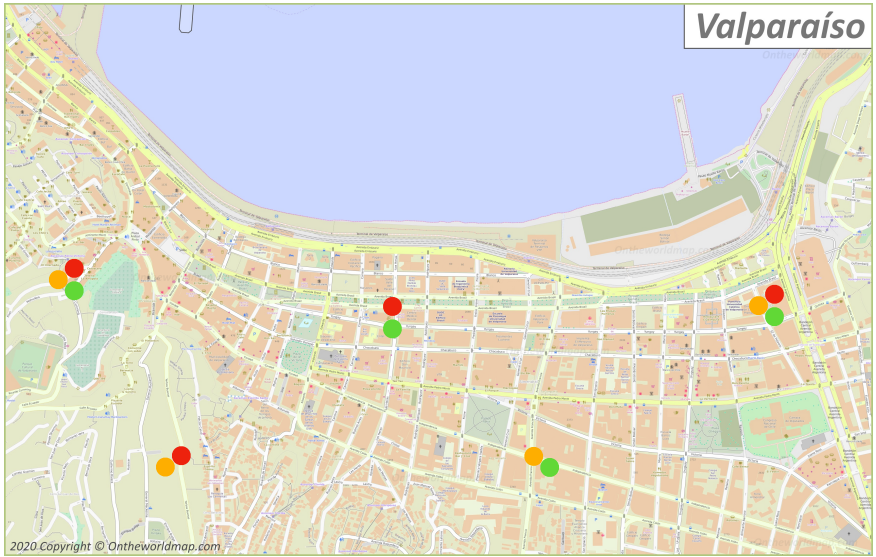
1. **Exponential term** with separate λ_i parameters
2. **Linear term** with fixed coupling

Connection to other methods

Method	Approach
Feature Selection	Discrete: keep or remove feature
Regularization (L1)	Shrink weights toward zero
ARD in Bayesian NNs	Prune weights/hyperparameters
ARD in GPs	Automatically scale input dimensions

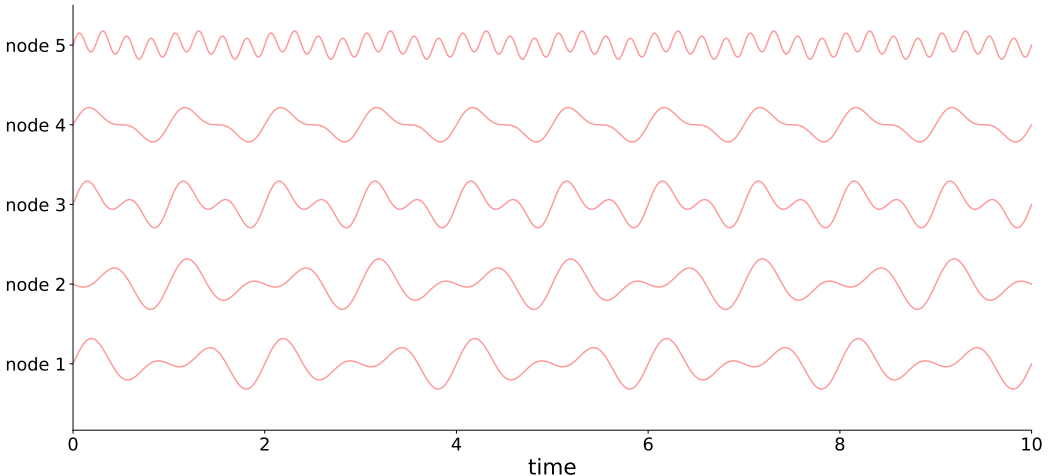
Multi-task or multi-output GPs

Multi-task GPs

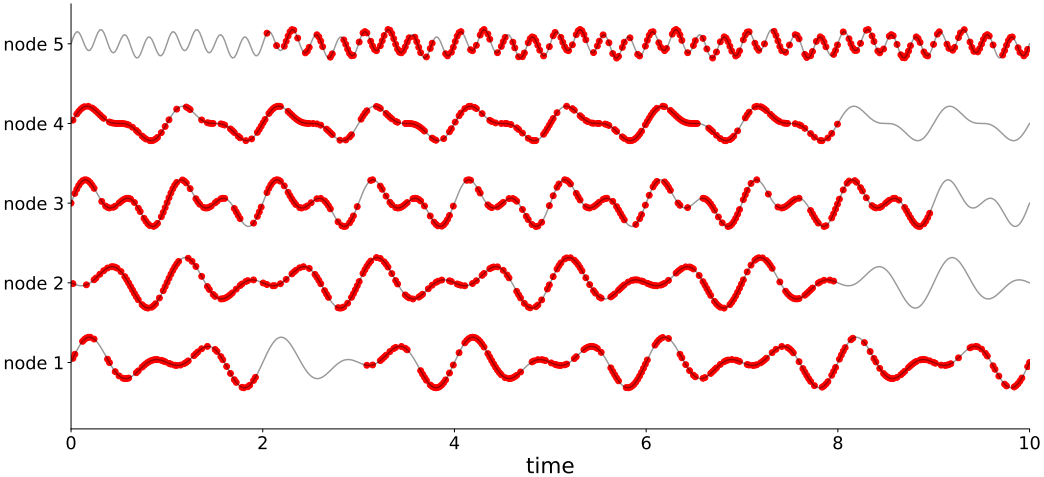


- Temperature
- Humidity
- CO2 level

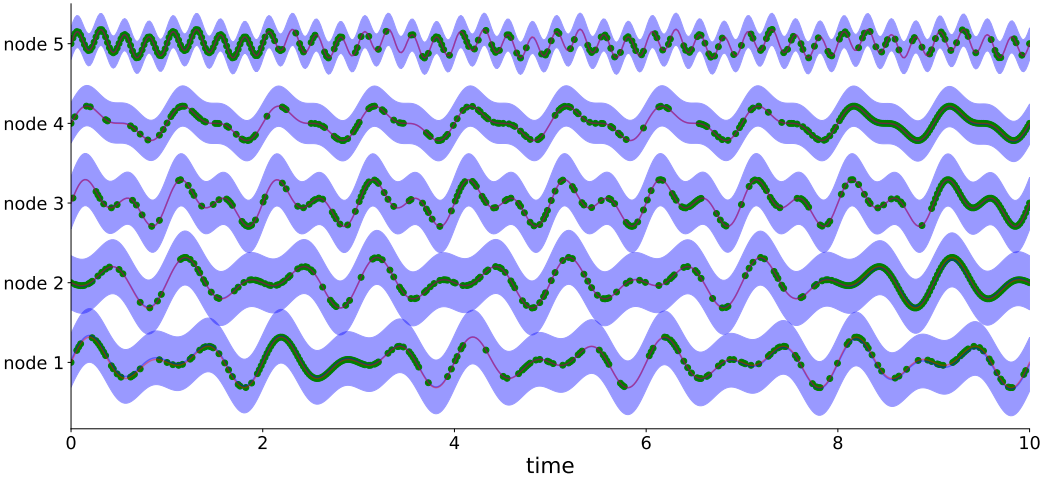
Sensors network



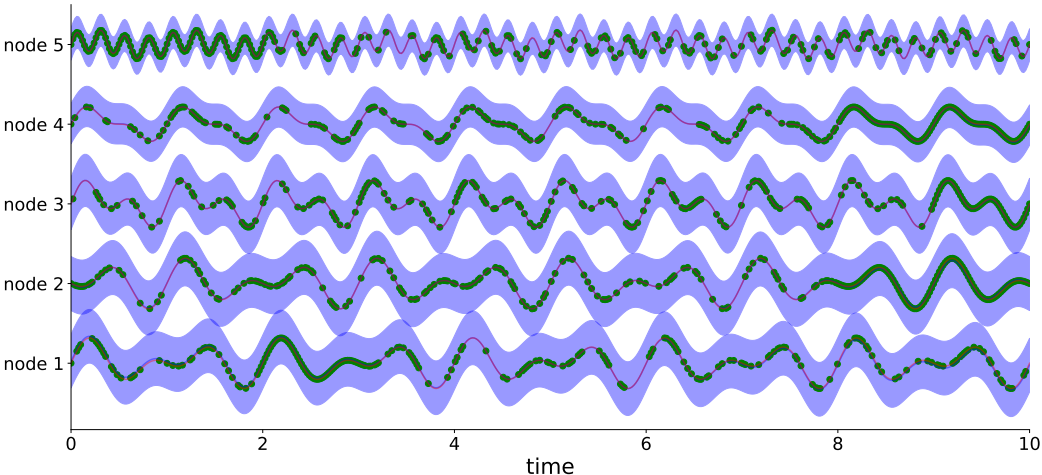
Sensors network with missing segments



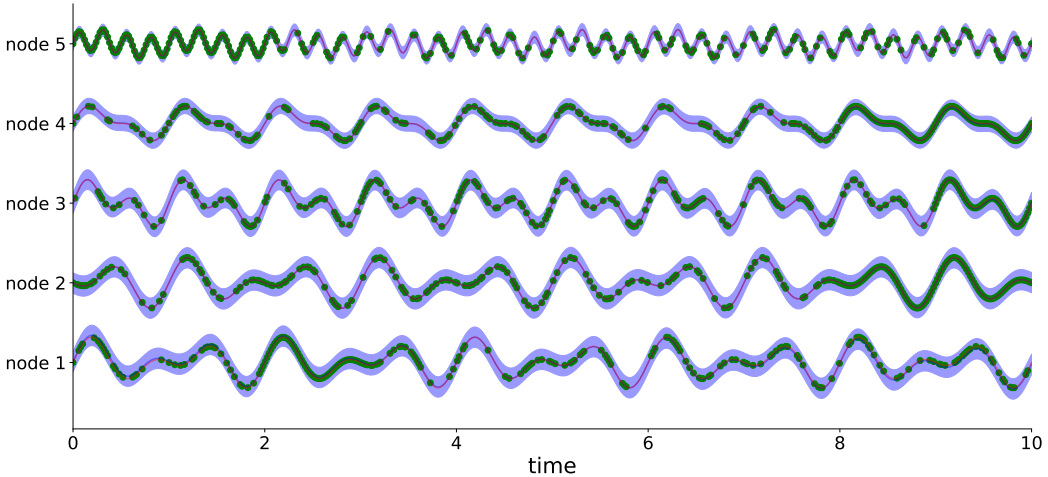
Sensors network - prediction



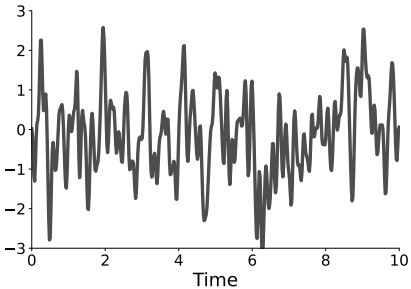
Sensors network - prediction before training



Sensors network - prediction after training

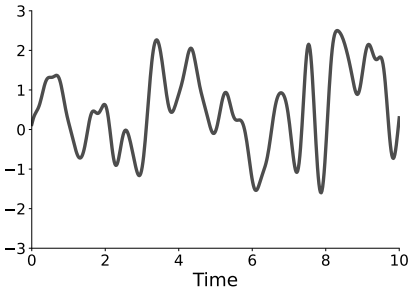


Single-output Gaussian process



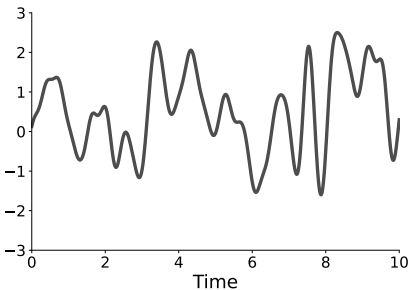
$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Single-output Gaussian process



$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Single-output Gaussian process

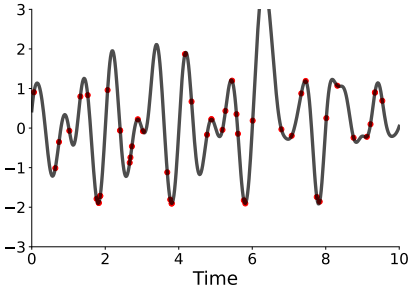


$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Training dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, f(\mathbf{x}_i)) \mid i = 1, \ldots, N\}$$

Single-output Gaussian process



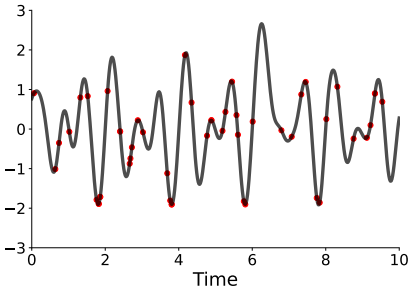
$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Training dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, f(\mathbf{x}_i)) \mid i = 1, \ldots, N\}$$

$$\begin{bmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}\right)$$

Single-output Gaussian process



$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

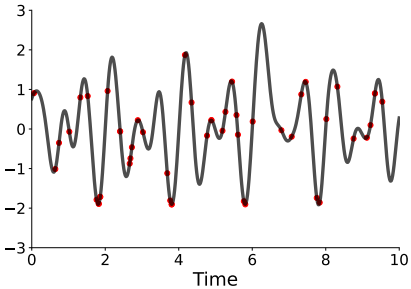
Training dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, f(\mathbf{x}_i)) \mid i = 1, \ldots, N\}$$

$$\begin{bmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}\right)$$

In matrix form: $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, \mathbf{K})$

Single-output Gaussian process



$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

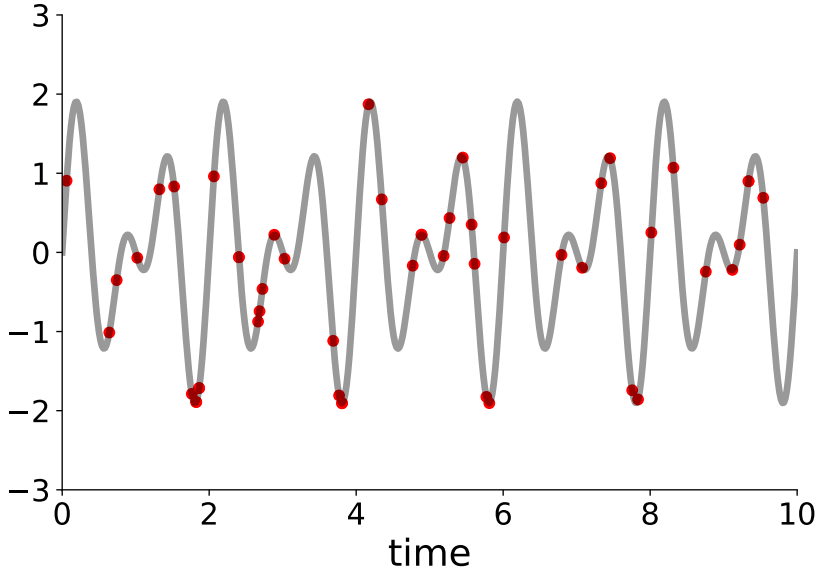
Training dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, f(\mathbf{x}_i)) \mid i = 1, \ldots, N\}$$

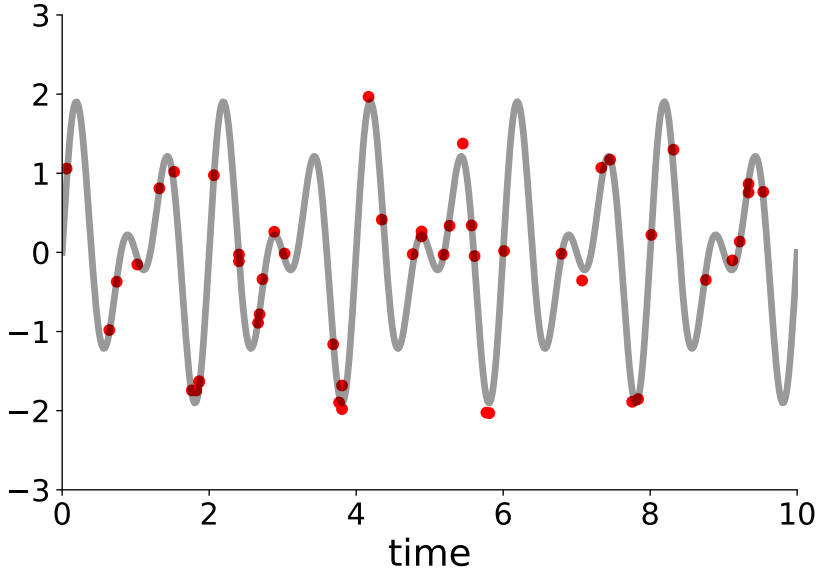
$$\begin{bmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}\right)$$

In matrix form: $\mathbf{f} \sim \mathcal{N}(\mathbf{0}, \mathbf{K})$ **For prediction:** $p(f(\mathbf{x}_*) \mid \mathbf{f})$

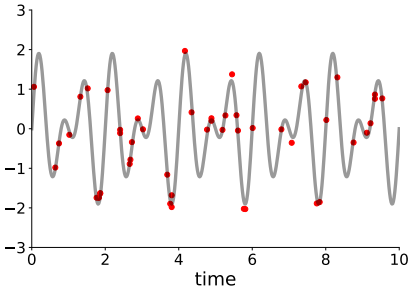
Noisy observations



Noisy observations

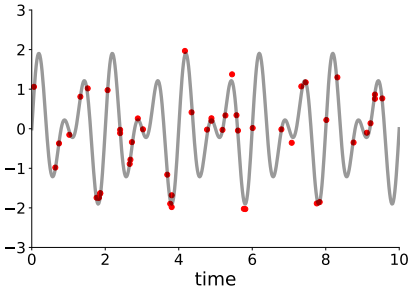


GP with noisy observations



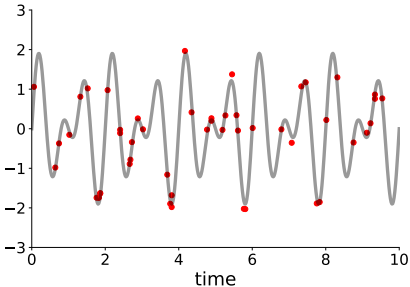
$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

GP with noisy observations



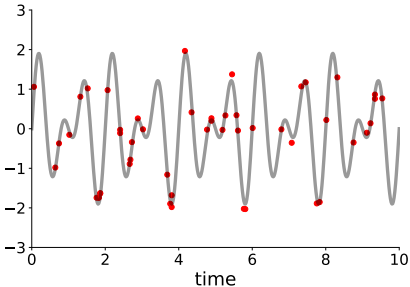
$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$
$$y\left(\mathbf{x}_i\right) = f\left(\mathbf{x}_i\right) + \epsilon_i$$

GP with noisy observations



$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$
$$y\left(\mathbf{x}_i\right) = f\left(\mathbf{x}_i\right) + \epsilon_i$$
$$\epsilon_i \sim \mathcal{N}\left(0, \sigma^2\right)$$

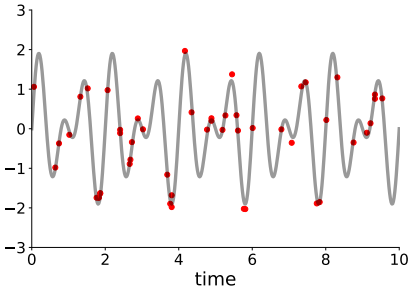
GP with noisy observations



$$
\begin{aligned}
f(\mathbf{x}) &\sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right) \\
y\left(\mathbf{x}_i\right) &= f\left(\mathbf{x}_i\right) + \epsilon_i \\
\epsilon_i &\sim \mathcal{N}\left(0, \sigma^2\right) \\
\mathcal{D} &= \left\{\left(\mathbf{x}_i, y\left(\mathbf{x}_i\right)\right) \mid i = 1, \ldots, N\right\}
\end{aligned}
$$

$$
\begin{bmatrix} y\left(\mathbf{x}_1\right) \\ \vdots \\ y\left(\mathbf{x}_N\right) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k\left(\mathbf{x}_1, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_1, \mathbf{x}_N\right) \\ \vdots & \ddots & \vdots \\ k\left(\mathbf{x}_N, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_N, \mathbf{x}_N\right) \end{bmatrix} + \sigma^2 \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix} \right)
$$

GP with noisy observations

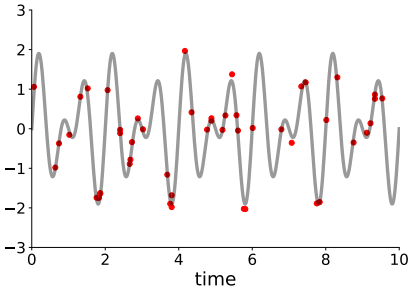


$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$
$$y\left(\mathbf{x}_i\right) = f\left(\mathbf{x}_i\right) + \epsilon_i$$
$$\epsilon_i \sim \mathcal{N}\left(0, \sigma^2\right)$$
$$\mathcal{D} = \left\{\left(\mathbf{x}_i, y\left(\mathbf{x}_i\right)\right) \mid i = 1, \ldots, N\right\}$$

$$\begin{bmatrix} y\left(\mathbf{x}_1\right) \\ \vdots \\ y\left(\mathbf{x}_N\right) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k\left(\mathbf{x}_1, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_1, \mathbf{x}_N\right) \\ \vdots & \ddots & \vdots \\ k\left(\mathbf{x}_N, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_N, \mathbf{x}_N\right) \end{bmatrix} + \sigma^2 \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}\right)$$

In matrix form: $\mathbf{f} \sim \mathcal{N}\left(\mathbf{0},\, \mathbf{K} + \sigma\mathbf{I}\right)$

GP with noisy observations

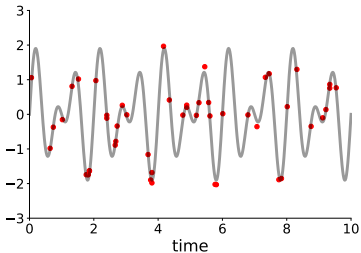


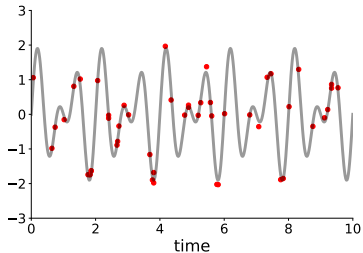
$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right)$$
$$y\left(\mathbf{x}_i\right) = f\left(\mathbf{x}_i\right) + \epsilon_i$$
$$\epsilon_i \sim \mathcal{N}\left(0, \sigma^2\right)$$
$$\mathcal{D} = \left\{\left(\mathbf{x}_i, y\left(\mathbf{x}_i\right)\right) \mid i = 1, \ldots, N\right\}$$

$$\begin{bmatrix} y\left(\mathbf{x}_1\right) \\ \vdots \\ y\left(\mathbf{x}_N\right) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k\left(\mathbf{x}_1, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_1, \mathbf{x}_N\right) \\ \vdots & \ddots & \vdots \\ k\left(\mathbf{x}_N, \mathbf{x}_1\right) & \cdots & k\left(\mathbf{x}_N, \mathbf{x}_N\right) \end{bmatrix} + \sigma^2 \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}\right)$$

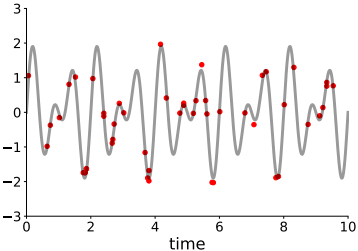
In matrix form: $\mathbf{f} \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K} + \sigma\mathbf{I}\right)$ **For prediction:** $p\left(f\left(\mathbf{x}_*\right) \mid \mathbf{y}\right)$

Multiple-output Gaussian process

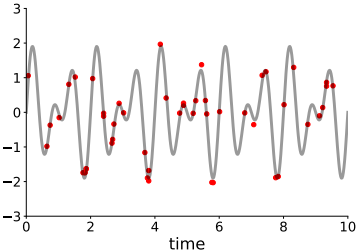




Multiple-output Gaussian process

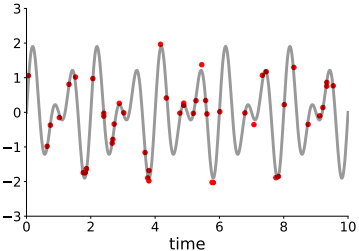


$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



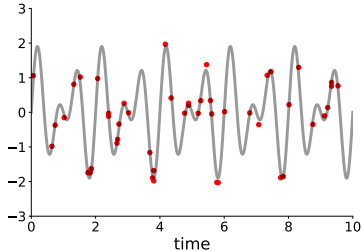
$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

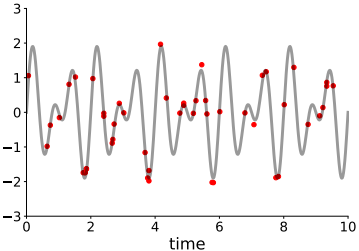
$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1}, f_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$



$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2}, f_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

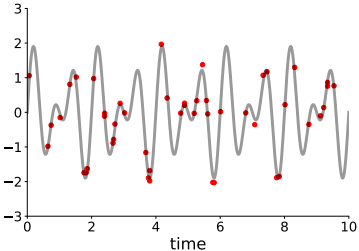
Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, f_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$

$$\mathbf{f}_1 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_1\right)$$

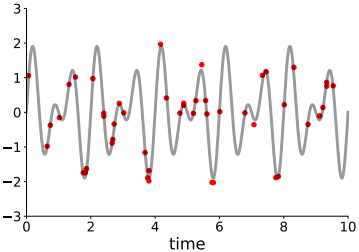


$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

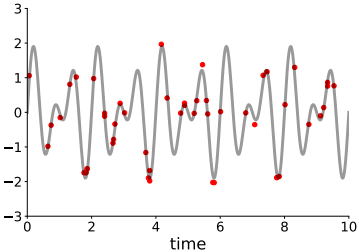
$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, f_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

$$\mathbf{f}_2 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_2\right)$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

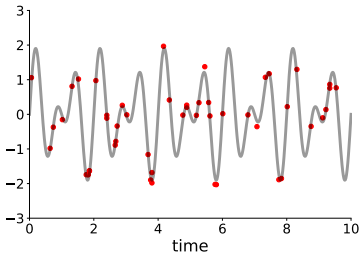
$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, f_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$

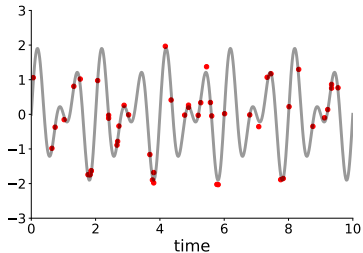
$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, f_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

$$\mathbf{f}_1 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_1\right) \qquad \mathbf{f}_2 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_2\right)$$

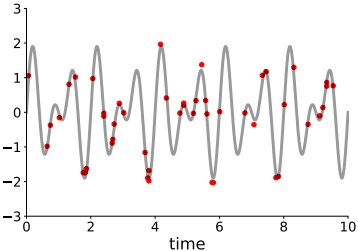
$$\underbrace{\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix}}_{\mathbf{f}} \sim \mathcal{N}\left(\underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}}_{\mathbf{0}}, \underbrace{\begin{bmatrix} \mathbf{K}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_2 \end{bmatrix}}_{\mathbf{K}_{\mathbf{f},\mathbf{f}}}\right)$$

Multiple-output Gaussian process

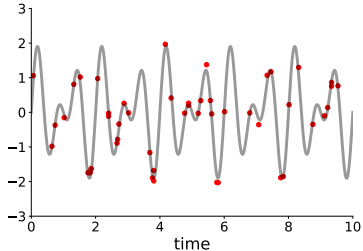




Multiple-output Gaussian process

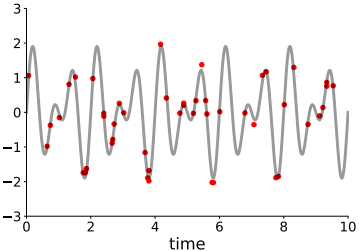


$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



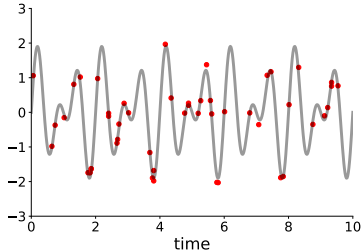
$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

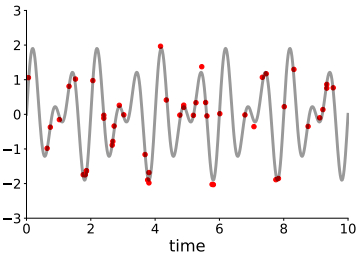
$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1}, y_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$



$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2}, y_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

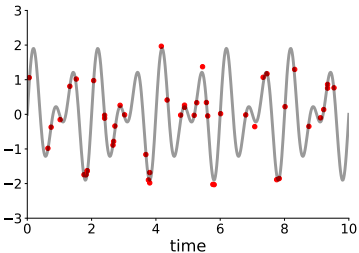
Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, y_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$

$$\mathbf{y}_1 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_1 + \sigma_1^2 \mathbf{I}\right)$$

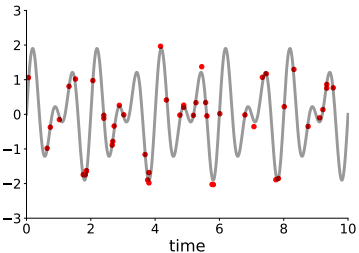


$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

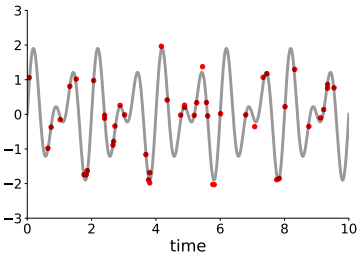
$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, y_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

$$\mathbf{f}_2 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_2 + \sigma_2^2 \mathbf{I}\right)$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

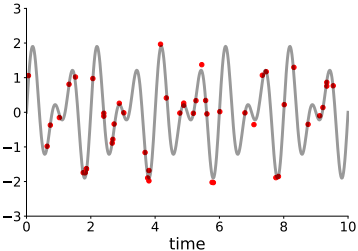
$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, y_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$

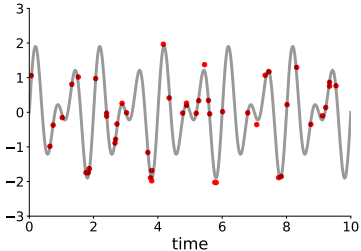
$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, y_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

$$\mathbf{y}_1 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_1 + \sigma_1^2 \mathbf{I}\right) \qquad \mathbf{f}_2 \sim \mathcal{N}\left(\mathbf{0}, \mathbf{K}_2 + \sigma_2^2 \mathbf{I}\right)$$

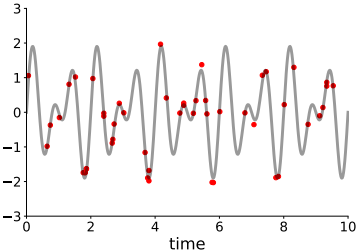
$$\underbrace{\begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix}}_{\mathbf{y}} \sim \mathcal{N}\left(\underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}}_{\mathbf{0}}, \underbrace{\begin{bmatrix} \mathbf{K}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_2 \end{bmatrix}}_{\mathbf{K}_{\mathbf{f},\mathbf{f}}} + \underbrace{\begin{bmatrix} \sigma_1^2 \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \sigma_2^2 \mathbf{I} \end{bmatrix}}_{\Sigma}\right)$$

Multiple-output Gaussian process

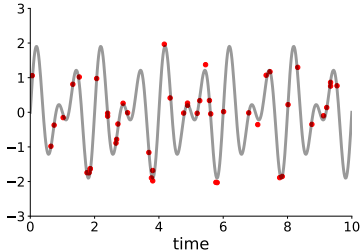




Multiple-output Gaussian process

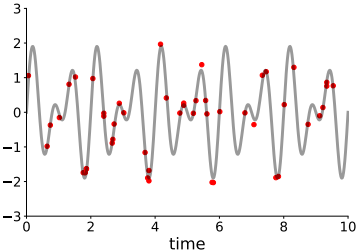


$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



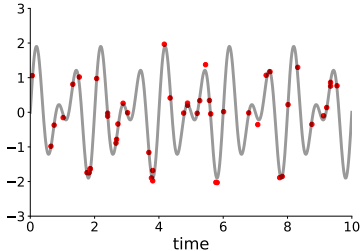
$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

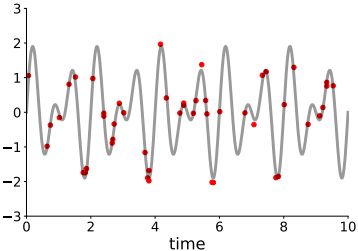
$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, f_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$



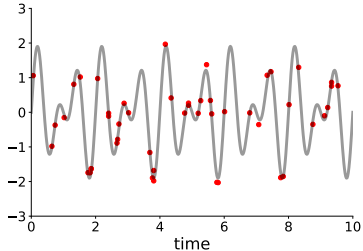
$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, f_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

Multiple-output Gaussian process



$$f_1(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_1\left(\mathbf{x}, \mathbf{x}'\right)\right)$$



$$f_2(\mathbf{x}) \sim \mathcal{GP}\left(0,\, k_2\left(\mathbf{x}, \mathbf{x}'\right)\right)$$

$$\mathcal{D}_1 = \left\{\left(\mathbf{x}_{i,1},\, f_1\left(\mathbf{x}_{i,1}\right)\right) \mid i = 1, \ldots, N_1\right\}$$

$$\mathcal{D}_2 = \left\{\left(\mathbf{x}_{i,2},\, f_2\left(\mathbf{x}_{i,2}\right)\right) \mid i = 1, \ldots, N_2\right\}$$

$$\mathbf{K}_{\mathbf{f},\mathbf{f}} = \begin{bmatrix} \mathbf{K}_1 & ? \\ ? & \mathbf{K}_2 \end{bmatrix}$$

Build a cross-covariance function $\text{cov}[f_1(\mathbf{x}),\, f_2(\mathbf{x}')]$ such that $\mathbf{K}_{\mathbf{f},\mathbf{f}}$ is positive semi-definite.

Why model task correlations?

Many applications: multi-output sensors, multi-channel time series, multi-label prediction.

If tasks are correlated, sharing structure can:

- Improve predictions for low-data tasks.
- Regularize hyperparameters and reduce overfitting.
- Provide coherent multi-output uncertainty.

Multi-task GPs

Multi-task Gaussian Processes (GPs) model multiple correlated outputs jointly.

Models:

- Intrinsic Coregionalization Model (ICM).
- Linear Model of Coregionalization (LMC).

The idea is to exploit task correlations to improve prediction and uncertainty quantification.

Vector-valued GP

Multi-task GP: $f : \mathcal{X} \to \mathbb{R}^D$, $f(x) = (f_1(x), \dots, f_D(x))^\intercal$.

Mean function:

$$m : \mathcal{X} \to \mathbb{R}^D.$$

Matrix-valued kernel:

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{D \times D},$$

encoding both input similarity and inter-task covariance.

Independent multi-task baseline

Simplest approach: one independent GP per task $i = 1, \dots, D$:

$$f_i(X) \sim \mathcal{N}\big(m_i(X), K_i(X, X)\big).$$

Joint covariance (stacking all tasks) is block-diagonal:

$$K(X, X) = \mathrm{diag}\big(K_1(X, X), \dots, K_D(X, X)\big).$$

No information sharing across tasks; correlations are ignored.

Intrinsic Coregionalization Model

Consider D outputs collected in $f(x) = (f_1(x), \dots, f_D(x))^\intercal$.

ICM assumes:

$$f(x) \sim \mathcal{GP}(0,\ K((x,i),(x',j))),$$

with separable covariance

$$\text{Cov}\left(f_i(x), f_j(x')\right) = B_{ij}\, k(x,x').$$

where $k(x,x')$ is a scalar positive definite kernel, and $B \in \mathbb{R}^{D\times D}$ is symmetric positive semidefinite.

ICM kernel as Kronecker product

Let $X = (x_1, \dots, x_N)$ and $K_X \in \mathbb{R}^{N \times N}$ with

$$(K_X)_{nm} = k(x_n, x_m).$$

Stack all outputs as $\text{vec}(F) \in \mathbb{R}^{ND}$, where $F_{n,i} = f_i(x_n)$.
Under ICM:

$$\text{vec}(F) \sim \mathcal{N}(0,\, K_{\text{ICM}}), \quad K_{\text{ICM}} = B \otimes K_X.$$

This shows a separable structure: task covariance $\times$ input covariance.

ICM: positive semidefiniteness of B

B must be positive semidefinite: $z^\mathsf{T} B z \geq 0$ for all $z \in \mathbb{R}^D$.

A common parameterization:

$$B = WW^\mathsf{T} + \mathsf{diag}(v),$$

with $W \in \mathbb{R}^{D \times R}$ (low rank) and $v \in \mathbb{R}^D_{\geq 0}$.

This guarantees $B \succeq 0$ and controls rank of inter-task correlations via R and marginal variances via v.

ICM: correlation coefficients

Covariance between tasks i, j at a fixed input:

$$\text{Cov}(f_i(x), f_j(x)) = B_{ij}\, k(x, x).$$

If $k(x, x) = \sigma_k^2$, the task covariance (up to σ_k^2) is B_{ij}.

Pearson correlation coefficient:

$$\rho_{ij} = \frac{B_{ij}}{\sqrt{B_{ii} B_{jj}}}.$$

ICM: latent-factor construction (rank-1)

Single latent GP:

$$u(x) \sim \mathcal{GP}(0, k(x, x')).$$

Linear mixing to outputs:

$$f_i(x) = a_i \, u(x), \quad i = 1, \dots, D.$$

Then

$$\text{Cov}(f_i(x), f_j(x')) = a_i a_j \, k(x, x'),$$

Hence $B = aa^{\mathsf{T}}$, which is rank-1 and positive semidefinite.

ICM: low-rank latent-factor view

Generalize to R latent GPs $u_r(x) \sim \mathcal{GP}(0, k(x, x'))$, independent over r.

Mixing:

$$f_i(x) = \sum_{r=1}^{R} w_{ir} u_r(x),$$

with $W = (w_{ir}) \in \mathbb{R}^{D \times R}$.

Then

$$\mathrm{Cov}(f_i(x), f_j(x')) = \sum_{r=1}^{R} w_{ir} w_{jr} k(x, x') = B_{ij} k(x, x'),$$

with $B = WW^{\mathsf{T}}$.

LMC: generalization

LMC allows multiple input kernels with latent GPs
$u_q(x) \sim \mathcal{GP}(0, k_q(x, x'))$, independent over $q = 1, \dots, Q$.

Mixing:

$$f_i(x) = \sum_{q=1}^{Q} a_{iq}\, u_q(x).$$

Then

$$\text{Cov}(f_i(x), f_j(x')) = \sum_{q=1}^{Q} a_{iq} a_{jq}\, k_q(x, x') = \sum_{q=1}^{Q} B_{ij}^{(q)} k_q(x, x'),$$

with $B^{(q)} = a_{:q} a_{:q}^{\mathsf{T}}$.

ICM as special case of LMC

If all latent processes share the same kernel k, i.e. $k_q = k$ for all q:

$$\text{Cov}(f_i(x), f_j(x')) = \left(\sum_{q=1}^{Q} B_{ij}^{(q)} \right) k(x, x') = B_{ij} k(x, x'),$$

where $B = \sum_{q=1}^{Q} B^{(q)}$.

- ICM typically refers to a single shared kernel and one coregionalization matrix, while LMC uses sums of such components.
- LMC can capture richer patterns when tasks behave differently across inputs.

Matrix form and structure

Under ICM, for N inputs and D tasks:

$$\text{vec}(F) \sim \mathcal{N}(0,\, B \otimes K_X).$$

Outputs are coupled via B and share smoothness through k.

Under LMC:

$$K_{\text{LMC}} = \sum_{q=1}^{Q} B^{(q)} \otimes K_X^{(q)},$$

with $(K_X^{(q)})_{nm} = k_q(x_n, x_m)$.

Noise modeling

Observations:

$$y(x) = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}_D(0, \Sigma),$$

with $\Sigma \in \mathbb{R}^{D \times D}$ positive semidefinite.

Choices: - Diagonal: $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_D^2)$. - Full: $\Sigma = LL^\intercal$ with $L \in \mathbb{R}^{D \times r}$.

Correlated noise models shared disturbances across tasks.

Joint covariance with noise

Let K be the noise-free covariance from ICM/LMC over all tasks and inputs.

For N inputs and D tasks (stacked), a common form is:

$$K_n = K + I_N \otimes \Sigma,$$

where I_N is the $N \times N$ identity matrix.

The Kronecker structure is preserved when Σ is diagonal; with full Σ structure is more complex but still interpretable.

Predictions and uncertainty

At test inputs X_*:

$$p(f(X_*) \mid Y) = \mathcal{N}(\mu_*, \Sigma_*).$$

For each task t:

- Predictive mean $\mu_{*,t}(x)$.
- Variance $\mathrm{Var}(f_t(x))$ for credible bands.

Joint Σ_* contains cross-task predictive covariances.

Learned B and Σ

Coregionalization matrix:

$$B = WW^{\mathsf{T}} + \mathrm{diag}(v),$$

learned from data.

Interpretation:

- B_{ii} is the marginal variance of task i.
- B_{ij} is the learned covariance between tasks i and j.

Noise covariance $\Sigma = LL^{\mathsf{T}}$ can also be inspected (e.g. heatmaps) to understand shared noise.

Independent GPs vs ICM

Independent GPs:

- Separate mean, kernel, and likelihood per task.
- No cross-task covariance; hyperparameters estimated from each task alone.

ICM:

- Shared input kernel hyperparameters.
- Cross-task structure encoded in B and possibly Σ.

With few observations per task, ICM can regularize and improve generalization.

When to use ICM vs LMC

ICM (single shared kernel):

- Tasks have similar smoothness and variation across input space.
- Fewer hyperparameters, simpler optimization.

LMC (multiple kernels):

- Tasks differ in smoothness, periodicity, or characteristic scales.
- More flexible but higher computational and optimization cost.

Practical strategy: Start with low-rank ICM; increase rank or move to LMC if cross-task structure appears underfitted.

Infinite Neural Network = Gaussian Process

Single Hidden Layer Architecture

One-Hidden-Layer Network

Network structure:

- Input: $\mathbf{x} \in \mathbb{R}^d$
- Hidden layer: J units producing $\mathbf{a}^{(1)} \in \mathbb{R}^J$
- Output layer: single linear unit producing $y_k \in \mathbb{R}$

Forward pass:

$$y_k = f(\mathbf{x}; \mathbf{w}) = \sum_{j=1}^{J} w_{kj}\, h\left(\sum_i w_{ji} x_i + w_{j0}\right) + w_{k0}$$

where:

- $h(\cdot)$ is the activation function (e.g., ReLU, tanh).
- w_{ji}, w_{j0}: weights and bias of hidden unit j.
- w_{kj}, w_{k0}: weights and bias of the single output neuron.

Data Flow: Inputs and Outputs

Component	Input(s)	Output
Single neuron	$\mathbf{x}$	scalar activation a
Single hidden layer	$\mathbf{x}$	vector $\mathbf{a}^{(1)} \in \mathbb{R}^J$
Whole finite NN	$\mathbf{x}$	scalar prediction $y_k = f(\mathbf{x}; \mathbf{w})$

Training

Weights $\mathbf{w}, \mathbf{w}_0$ are **parameters** adjusted by backpropagation and gradient descent

From deterministic to Bayesian

Bayesian view (Neal 1994)

Weights become **random variables** with a prior distribution:

Output layer weights:

$$w_{1j} \sim \mathcal{N}\left(0, \frac{\alpha}{J}\right), \quad w_{10} \sim \mathcal{N}(0, \sigma^2)$$

Hidden layer weights:

$$w_{ji} \sim \mathcal{N}(0, \sigma_w^2), \quad w_{j0} \sim \mathcal{N}(0, \sigma_b^2)$$

From deterministic to Bayesian

What Changes?

In the Bayesian view, the **network output becomes a random variable**:

$$f(\mathbf{x}) \sim ?$$

- The **input** is a fixed point $\mathbf{x}$
- **Randomness** is added by random weights
- The **output** is a random scalar $f(\mathbf{x})$

Central Limit Theorem

For any fixed finite set of inputs $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}\}$:

The vector of outputs:

$$\big(f(\mathbf{x}^{(1)}), \ldots, f(\mathbf{x}^{(N)})\big)$$

can be written as a sum of J i.i.d. random vectors (one contribution per hidden unit).

By the multivariate Central Limit Theorem:

$$\lim_{J \to \infty} \big(f(\mathbf{x}^{(1)}), \ldots, f(\mathbf{x}^{(N)})\big) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

where $\boldsymbol{\mu} = \mathbf{0}$ and $\boldsymbol{\Sigma}_{ij} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

Infinite Neural Network = Gaussian Process

Theorem (Neal 1994)

When a one-hidden-layer Bayesian neural network has $J \to \infty$ hidden units with i.i.d. zero-mean weight priors (properly scaled), the random function converges in distribution to a Gaussian Process:

$$f(\cdot) \sim \mathcal{GP}\left(0, k(\mathbf{x}, \mathbf{x}') = \alpha\, \mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right) h\left(\sum_i w_{ji} x'_i + w_{j0}\right)\right] + \sigma^2\right)$$

Mean Function

$$
\begin{aligned}
m(\mathbf{x}) &= \mathbb{E}\left[\sum_{j=1}^{J} w_{1j}\, h\left(\sum_i w_{ji} x_i + w_{j0}\right) + w_{10}\right] \\
&= \sum_{j=1}^{J} \mathbb{E}[w_{1j}] \cdot \mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right)\right] + \mathbb{E}[w_{10}] \\
&= \sum_{j=1}^{J} 0 \cdot \mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right)\right] + 0 \\
&= 0
\end{aligned}
$$

where we used that all weights have **zero mean**.

Covariance Function

For any two distinct inputs $\mathbf{x}, \mathbf{x}'$:

$$k(\mathbf{x}, \mathbf{x}') = \text{Cov}\left[f(\mathbf{x}), f(\mathbf{x}')\right] = \mathbb{E}\left[f(\mathbf{x}) \cdot f(\mathbf{x}')\right]$$

(since $\mathbb{E}[f(\mathbf{x})] = 0$)

Covariance Function Derivation

$$
\begin{aligned}
k(\mathbf{x}, \mathbf{x}') &= \mathbb{E}\left[\left(\sum_{j=1}^{J} w_{1j} h\left(\sum_i w_{ji} x_i + w_{j0}\right) + w_{10}\right)\left(\sum_{j=1}^{J} w_{1j} h\left(\sum_i w_{ji} x'_i + w_{j0}\right) + w_{10}\right)\right] \\
&= \mathbb{E}\left[\sum_{j=1}^{J} w_{1j}^2\, h\left(\sum_i w_{ji} x_i + w_{j0}\right) h\left(\sum_i w_{ji} x'_i + w_{j0}\right)\right] + \mathbb{E}[w_{10}^2] \\
&= \sum_{j=1}^{J} \mathbb{E}[w_{1j}^2]\mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right) h\left(\sum_i w_{ji} x'_i + w_{j0}\right)\right] + \mathsf{Var}(w_{10}) \\
&= \sum_{j=1}^{J} \frac{\alpha}{J}\, \mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right) h\left(\sum_i w_{ji} x'_i + w_{j0}\right)\right] + \sigma^2 \\
&= \alpha\, \mathbb{E}\left[h\left(\sum_i w_{ji} x_i + w_{j0}\right) h\left(\sum_i w_{ji} x'_i + w_{j0}\right)\right] + \sigma^2
\end{aligned}
$$

Extension to Deep Networks

Published as a conference paper at ICLR 2018

DEEP NEURAL NETWORKS AS GAUSSIAN PROCESSES

Jaehoon Lee[*†], **Yasaman Bahri**[*†], **Roman Novak**, **Samuel S. Schoenholz**, **Jeffrey Pennington**, **Jascha Sohl-Dickstein**

Google Brain
{jaehlee, yasamanb, romann, schsam, jpennin, jaschasd}@google.com

Extension to Deep Networks

Single hidden layer to multiple layers

The Central Limit Theorem argument extends by **induction** over layers. If the input to layer ℓ is governed by a GP, then:

- The output of layer ℓ is a **sum of independent terms** (one per neuron).
- By CLT, it is also a GP.
- This applies recursively to all layers.

Result: An **infinitely wide deep network** (all layers with $N_\ell \to \infty$) is a Gaussian Process.

Recursive kernel computation

Building kernels layer by layer

For a deep network with L hidden layers, denote:

- $z^{(\ell)}(x)$: output of layer ℓ (pre-activation of next layer)
- $x^{(\ell)}(x)$: activations of layer ℓ (post-nonlinearity)

Layer ℓ output:

$$z_i^{(\ell)}(x) = b_i^{(\ell)} + \sum_{j=1}^{N_\ell} W_{ij}^{(\ell)} x_j^{(\ell-1)}(x)$$

where $x_j^{(\ell-1)}(x) = \varphi(z_j^{(\ell-1)}(x))$ (activation function φ).

Recursive kernel formula

Base case (input layer):

$$K^{(0)}(x, x') = \mathbb{E}[z^{(0)}(x) z^{(0)}(x')]$$

For raw inputs:

$$K^{(0)}(x, x') = \sigma_b^2 + \sigma_w^2 \frac{\mathbf{x} \cdot \mathbf{x}'}{d_{in}}$$

where σ_w^2, σ_b^2 are weight and bias variances, d_{in} is input dimension.

Recursive kernel: inductive step

Assume layer $\ell - 1$ computes a GP with kernel $K^{(\ell-1)}$.

For layer ℓ, the output is:

$$z_i^{(\ell)}(x) = b_i^{(\ell)} + \sum_{j=1}^{N_\ell} W_{ij}^{(\ell)} \varphi(z_j^{(\ell-1)}(x))$$

As $N_\ell \to \infty$, by CLT:

$$z^{(\ell)}(x) \sim \mathcal{GP}(0, K^{(\ell)})$$

with kernel:

$$K^{(\ell)}(x, x') = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{z^{(\ell-1)} \sim \mathcal{GP}(0, K^{(\ell-1)})} \left[\varphi(z^{(\ell-1)}(x)) \, \varphi(z^{(\ell-1)}(x')) \right]$$

Gaussian Process Deconvolution

Gaussian Process Deconvolution

Application	Domain
De-reverberation	Acoustic signal processing
Super-resolution	Image restoration, low-resolution to high-resolution
Perfusion imaging	fMRI, medical imaging (hemodynamic response)
Seismic imaging	Geophysics, signal propagation
Astronomy	Point-spread function deconvolution

The Deconvolution Problem

We observe noisy, possibly incomplete measurements y of a convolved signal:

$$y(t) = \int h(\tau)x(t-\tau)d\tau + \epsilon(t)$$

where:

- $x(t)$ is the latent source signal.
- $h(\tau)$ is the convolution filter (known or unknown).
- $\epsilon(t)$ is the measurement noise.
- $y(t)$ is the observed signal

Challenges

Traditional deconvolution methods (Wiener, Inverse FT) often suffer from:

- **Numerical instability** when filter has low spectral power.
- **No uncertainty quantification** (only point estimates).
- **Assumptions on signal structure** that may be unrealistic.

The GP Solution

Place a **Gaussian process prior** on the latent source $x(t)$.

The GP Hierarchical Model:

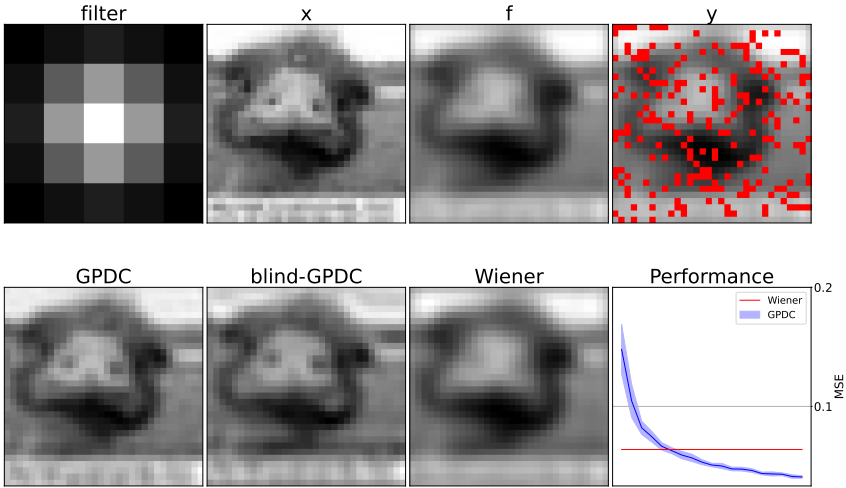
Given GP prior: $x(t) \sim \mathcal{GP}(m(t), k(t, t'))$

The posterior deconvolution is:

$$\hat{x}|y \sim \mathcal{GP}(\hat{m}(t), \hat{k}(t, t'))$$

where covariances are derived through convolution operators applied to the kernel k.

Example result



Resources

Rasmussen, Carl Edward. Gaussian processes in machine learning. Springer, Berlin, Heidelberg, 2003.

Richard Turner lecture notes

Mauricio Álvarez lecture notes

Gaussian processes in Python - scikit-learn.

Gaussian processes in Python - GPy.

Gaussian processes in Python - Gpytorch.

Gaussian process deconvolution (GAMES-UChile)